

Clocking Wizard v1.0 for Versal ACAP

LogiCORE IP Product Guide

Vivado Design Suite

PG321 (v1.0) October 26, 2022

Xilinx is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	6
About the Core.....	6
Recommended Design Experience.....	7
Feature Summary.....	7
Applications.....	8
Licensing and Ordering.....	8
Chapter 3: Product Specification.....	9
Performance.....	9
Resource Use.....	10
Port Descriptions.....	10
Register Space.....	14
Chapter 4: Designing with the Core.....	17
General Design Guidelines.....	17
Clocking.....	17
Resets.....	17
Functional Overview.....	18
Chapter 5: Design Flow Steps.....	27
Customizing and Generating the Core.....	27
Constraining the Core.....	60
Simulation.....	61
Chapter 6: Detailed Example Design.....	63
Unsupported Features in Example Design.....	63
Directory and File Contents.....	63

Chapter 7: Test Bench.....	65
Appendix A: Migrating.....	66
Differences between the Clocking Wizard and the Legacy DCM and PLL Wizards.....	66
Appendix B: Debugging.....	69
Finding Help on Xilinx.com.....	69
Debug Tools.....	70
Hardware Debug.....	71
Appendix C: Additional Resources and Legal Notices.....	72
Xilinx Resources.....	72
References.....	72
Training Resources.....	72
Revision History.....	73
Please Read: Important Legal Notices.....	74

Introduction

The LogiCORE™ IP Clocking Wizard core simplifies the creation of HDL source code wrappers for clock circuits customized to your clocking requirements. The Clocking Wizard guides you in setting the appropriate attributes for your clocking primitive, and allows you to override any parameter calculated by the wizard. In addition to providing a HDL wrapper for implementing the desired clocking circuit, the Clocking Wizard can also guide you in choosing the appropriate clocking primitive based on your requirements.

Features

The Clocking Wizard for Versal® adaptive compute acceleration platforms (ACAPs):

- Offers a mixed-mode clock manager (MMCM) and different phase-locked loop (XPLL/DPLL) primitives. GUI options must be enabled for using supported primitive features.
- Provides guidance for selecting the appropriate clocking primitive based on your requirements.
- Accepts up to two input clocks and up to seven output clocks per clock network.
- Provides AXI4-Lite interface for dynamically reconfiguring the clocking primitives for multiply, divide, phase shift/offset, or duty cycle programmability.
- Automatically calculates voltage-controlled oscillator (VCO) frequency for primitives with an oscillator, and provides multiply and divide values based on input and output frequency requirements.
- Automatically implements overall configuration that supports phase shift and duty cycle requirements.
- Inserts the buffer needed for input and output clocks.
- Provides the ability to override the selected clock primitive attributes.

Note: The user should be cautious when override mode is selected, and ensure all the overwritten values will fall into valid values and a valid VCO/PFD frequencies range.

- Provides a synthesizable example design including the clocking network and a simulation test bench.
- Provides optional ports for the selected primitive.

- Input and output ports can be renamed as per the user's requirements.

IP Facts

LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	Versal® ACAP
Supported User Interfaces	AXI4-Lite
Resources	Resource Use
Special Features	MMCM, XPLL, DPLL, Auto, Primitive Override, Spread Spectrum, Dynamic Reconfiguration, Clock Monitor, Auto Buffer, Clock Grouping, Safe Clock Startup, Analog/Digital deskew and MBUFGCE
Provided with Core	
Design Files	Verilog ²
Example Design	Verilog
Test Bench	Verilog ²
Constraints File	Xilinx® Design Constraints
Simulation Model	For supported simulators, see the Xilinx Design Tools: Release Notes Guide
Instantiation Template	Verilog and VHDL Wrapper
Supported S/W Driver ²	Not Applicable
Tested Design Flows ³	
Design Entry	Vivado® Design Suite
Simulation	Mentor Graphics QuestaSim, Vivado Simulator. For supported simulators, see the Xilinx Design Tools: Release Notes Guide .
Synthesis	Vivado Synthesis
Support	
Xilinx Support web page	
Release Notes and Known Issues	Master Answer Record: AR 76369
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775

Notes:

1. For a complete list of supported devices, see the Vivado IP catalog.
2. Standalone driver details can be found in the Vitis directory (<install_directory>/Vitis/<release>/data/embeddedsw/doc/xilinx_drivers.htm). Linux OS and driver support information is available [Xilinx Wiki](#) page.
3. For the supported versions of the tools, see the [Xilinx Design Tools: Release Notes Guide](#).

Overview

Navigating Content by Design Process

Xilinx® documentation is organized around a set of standard design processes to help you find relevant content for your current development task. All Versal® ACAP design process [Design Hubs](#) and the [Design Flow Assistant](#) materials can be found on the [Xilinx.com](#) website. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the Vivado® timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Register Space](#)
 - [Clocking and Resets](#)
 - [Customizing and Generating the Core](#)
 - [Chapter 6: Detailed Example Design](#)
-

About the Core

This chapter introduces the Clocking Wizard core and provides related information, recommended design experience, additional resources, technical support, and ways of submitting feedback to Xilinx. The Clocking Wizard core generates source register transfer level (RTL) code to implement a clocking network matched to your requirements. Both Verilog and VHDL design instantiation templates are supported.

The Clocking Wizard core is a Xilinx® IP core that can be generated using the Xilinx Vivado® design tools, included with the latest Vivado Design Suite release in the Xilinx Download Center. The core is licensed under the terms of the Xilinx End User License and no FLEX license key is required.

Recommended Design Experience

The Clocking Wizard is designed for users with any level of experience. The wizard automates the process of creating a clocking network by guiding you to the proper primitive configuration and allowing advanced users to override and manually set any attribute. Although the Clocking Wizard provides a fully verified clocking network, understanding the Xilinx® clocking primitives will aid you in making design trade-off decisions.

Feature Summary

Clocking features include:

- **Spread Spectrum:** This feature provides modulated output clocks which reduce the spectral density of the electromagnetic interference (EMI) generated by electronic devices. This feature is available only for the mixed-mode clock manager (MMCM) primitive. There is no support of UNISIM simulation for this feature. The Minimize Power and Dynamic Reconfiguration features are not available when Spread Spectrum is selected.
- **Phase Alignment:** This feature allows the output clock to be phase-locked to a reference, such as the input clock pin for a device. This feature is enabled by default.
- **Minimize Power:** This feature minimizes the amount of power needed for the primitive at the possible expense of frequency, phase offset, or duty cycle accuracy.
- **Dynamic Phase Shift:** This feature allows you to change the phase relationship on the output clocks.
- **Dynamic reconfiguration:** This feature allows you to change the programming of the primitive after device configuration. When this option is chosen, AXI4-Lite interface is selected by default for reconfiguring the clocking primitive. See [Dynamic Reconfiguration through AXI4-Lite](#) for more information.
- **Jitter Options:**
 - **Balanced:** Selecting Balanced results in the software choosing the correct bandwidth for jitter optimization.
 - **Minimize Output Jitter:** This feature minimizes the jitter on the output clocks, but at the expense of power and possibly output clock phase error. This feature is not available with Maximize Input Jitter Filtering.
 - **Maximize Input Jitter Filtering:** This feature allows for larger input jitter on the input clocks, but can negatively impact the jitter on the output clocks. This feature is not available with Minimize Output Jitter.

- **Auto Primitive:** Selecting this primitive instantiates the appropriate clocking primitive for your requirements. The wizard sets up the actual primitive automatically based on your inputs.
- **Clock Monitor:** This feature helps to monitor the clock inputs of the clocking wizard. It can monitor upto four clocks. It helps to monitor the input frequency is out of the range, detects clock stop and clock glitches.
- **Auto Buffer selection:** When `Clkout_drives` is selected as Buffer which means it is auto Buffer and IP will optimally generate the buffer (BUFG or MBUFGCE) by considering all other user inputs. When `Clkout_drives` is selected as "Buffer with CE" which is also auto buffer, and IP will optimally generate the buffer (either BUFGCE or MBUFGCE) by considering remaining user inputs.
- **Clock Grouping:** This grouping will be useful when user needs set of clocks under same group (that is derivable from same clock root). Based on clock grouping and auto buffer, IP will optimally derive BUFG or MBUFGCE. This feature will be useful even in the case of normal mode (non-auto buffer mode) for selection of common clocks under same group.
- **Safe Clock Startup:** This feature is used to get a stable and valid clock at the output. Sequencing is the additional option that helps to configure the clock outputs in the sequence that the user requested.

Applications

The Clocking Wizard core is used in applications such as:

- The creation of clock networks with the required frequency, phase, and duty cycle, with reduced jitter.
- Electromagnetic interference reduction in electronic devices using the Spread Spectrum feature.
- When dynamically reconfigured to different set of frequencies.
- To monitor the clocks for ensuring the clocks are stable and do not have a glitch.

Licensing and Ordering

This Xilinx® LogiCORE™ IP module is provided at no additional cost with the Xilinx Vivado® Design Suite under the terms of the [Xilinx End User License](#).

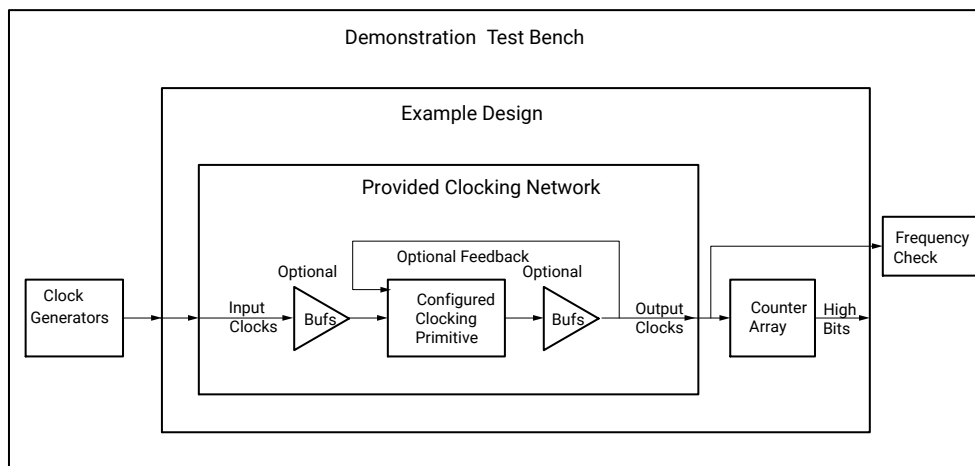
Information about other Xilinx® LogiCORE™ IP modules is available at the [Xilinx Intellectual Property](#) page. For information about pricing and availability of other Xilinx LogiCORE IP modules and tools, contact your [local Xilinx sales representative](#).

Product Specification

Clocking Wizard helps create the clocking circuit for the required output clock frequency, phase and duty cycle using mixed-mode clock manager (MMCM) or phase-locked loop (XPLL/DPLL) primitive. It also helps verify the generated output clock frequency in simulation, providing a synthesizable example design which can be tested on the hardware.

The functional block diagram of the core is shown in the following figure.

Figure 1: Core Block Diagram



X21856 110918

Performance

Power

- Minimize power feature minimizes the amount of power needed for the primitive at the possible expense of frequency, phase offset, or duty cycle accuracy.
- Power Down input pin, when asserted, places the clocking primitive into low power state, which stops the output clocks.

Resource Use

Resource usage is available in the Clocking Wizard GUI. To view the resource use details, click the Resource tab.

Figure 2: Resources

Clocking Wizard (1.0)

Documentation IP Location Switch to Defaults

IP Symbol **Resource**

PRIMITIVE is: MMCM

1 IBUF

1 BUFG

Component Name clk_wizard_0

Clocking Features Output Clocks MMCM Settings Optional Ports Summary

Clock Monitor

☐ Enable Clock Monitoring

Select Clocking Primitive

Primitive MMCM

Clocking Features

☐ Minimize Power ☐ Spread Spectrum

☐ Phase Alignment ☐ Dynamic Reconfig

☐ Safe Clock Startup

Jitter Options

Jitter Optimization Balanced

Interface Selection

☒ AXI4Lite ☐ APB3

Input Clock Information

Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒ Primary	clk_in1	100.000	10.000 - 680.000	UI	0.010	Single ended clock
☐ Secondary	clk_in2	100.000	72.000 - 144.000		0.010	Single ended clock

OK Cancel

Port Descriptions

The following table describes the input and output ports provided from the clocking network. All ports are optional, with the exception being that at least one input and one output clock are required. The options selected determine which ports are actually available to be configured. For example, when PI Control is selected as Fine_PS, the pscclk, psen, psincdec, and psdone ports are exposed. Any port that is not exposed is appropriately tied off or connected to a signal labeled unused in the delivered source code.

Input Clock Ports

Table 1: Clocking Wizard Input Clock Ports

Port Name	I/O	Description
clk_in1	Input	Clock in 1: Single-ended primary input clock port. Available when single-ended primary clock source is selected.
clk_in1_p	Input	Clock in 1 Positive and Negative: Differential primary input clock port pair. Available when a differential primary clock source is selected.
clk_in1_n		
clk_in2 ¹	Input	Clock in 2: Single-ended secondary input clock port. Available when a single-ended secondary clock source is selected.
clk_in2_p ¹	Input	Clock in 2 Positive and Negative: Differential secondary input clock port pair. Available when a differential secondary clock source is selected.
clk_in2_n ¹		
clk_in_sel ¹	Input	Clock in Select: When set to 1, the secondary input clock is enabled and when set to 0, the primary input clock is enabled. Note: This port is available only when the secondary clock is enabled using GUI.
clkfb_in	Input	Clock Feedback in: Single-ended feedback in port of the clocking primitive. Available when user-controlled on-chip, user controller-off chip, or automatic control off-chip feedback option is selected.
clkfb_in_p	Input	Clock Feedback in: Positive and Negative: Differential feedback in port of the clocking primitive. Available when the automatic control off-chip feedback and differential feedback option is selected.
clkfb_in_n		
clkin1_deskew	Input	Clock in for DESKEW PD1 Network: Single ended clock in deskew port is available when DESKEW1_IN is selected as "clk_ext" if any of the clock out (clk_out1 to clk_out7) PI Control is selected as DESKEW_PD1
clkin2_deskew	Input	Clock in for DESKEW PD2 Network: Single ended clock in deskew port is available when DESKEW2_IN is selected as "clk_ext" if any of the clock out (clk_out1 to clk_out7) PI Control is selected as DESKEW_PD2
clkfb1_deskew	Input	Clock fb for DESKEW PD1 Network: Single ended clock fb deskew port is available when any of the clock out (clk_out2 to clk_out7) PI Control is selected as DESKEW_PD1 and drives is selected as No_buffer for that particular clock and Deskew1 FB value in Deskew Network pane selected as per the clk_out (2 for clk_out2, 3 for clk_out3 etc)
clkfb2_deskew	Input	Clock fb for DESKEW PD2 Network: Single ended clock fb deskew port is available when any of the clock out (clk_out2 to clk_out7) PI Control is selected as DESKEW_PD2 and drives is selected as No_buffer for that particular clock and Deskew2 FB value in Deskew Network pane selected as per the clk_out (2 for clk_out2, 3 for clk_out3 etc)
ref_clk	Input	Reference clock: This is the input reference clock used to monitor the user clocks. It is considered to be stable and error free.
user_clk0	Input	User Clock0: This port is disabled when the ENABLE_DPLL/PLL/MMCM0 checkbox is enabled in the Vivado IDE. This option is enabled when you select the Clock Monitor.
user_clk1	Input	User Clock1: This port is disabled when the ENABLE_DPLL/PLL/MMCM1 checkbox is enabled in the Vivado IDE. This option is enabled when you select Clock Monitor.
user_clk2	Input	User Clock2: User input clock 2 to monitor. This option is enabled when you select Clock Monitor.

Table 1: Clocking Wizard Input Clock Ports (cont'd)

Port Name	I/O	Description
user_clk3	Input	User Clock3: User input clock 3 to monitor. This option is enabled when you select Clock Monitor.

Notes:

1. The `clk_in2`, `clk_in_sel`, `clk_fb_in*` pins are not available when the PLL or DPLL primitives are chosen.
2. At least one input clock is required; any design has at least a `clk_in1` or a `clk_in1_p/clk_in1_n` port.

Output Clock Ports

Table 2: Clocking Wizard Output Clock Ports

Port Name	I/O	Description
clk_out1	Output	Clock Out 1: Output clock of the clocking network. <code>clk_out1</code> is not optional.
clk_out1_ce	Input	Clock Enable: Clock enable pin of the output buffer. Available when the BUFGCE or BUFGCE_DIV or BUFG_GT or MBUFGCE buffers are used as output clock drivers.
clk_out1_clr	Input	Counter reset for divided clock output: Available when the BUFGCE_DIV or BUFG_GT buffer is used as an output clock driver. <code>clk_out1_clr</code> is active-High signal.
clk_out1_clr_n	Input	Counter reset for divided clock output: Available when the MBUFGCE buffer is used as output clock driver. It is an active-Low signal.
clk_out[2-n]_ce	Input	Clock Enable: Clock enable pin of the output buffer. Available when the BUFGCE or BUFGCE_DIV or BUFG_GT or MBUFGCE buffers are used as output clock drivers.
clk_out[2-n]_clr_n	Input	Counter reset for divided clock output: Available when the MBUFGCE buffer is used as output clock driver. It is an active-Low signal.
clk_out[2-n]_clr	Input	Counter reset for divided clock output: Available when BUFGCE_DIV buffer is used as output clock driver. <code>clk_out [2-n]_clr</code> is active-High signal.
clkfb_out	Output	Clock Feedback Out: Single ended feedback port of the clocking primitive. Available when the user-controlled feedback or automatic control off chip with single ended feedback option is selected.
clkfb_out_p	Output	Clock Feedback Out: Positive and Negative: Differential feedback output port of the clocking primitive. Available when the user-controlled off-chip feedback and differential feedback option is selected.
clkfb_out_n		
clk[1-7]_clr_n	Input	Clock CLR: Clock clear pin. Available when the buffer (auto buffer, either "Buffer" or "Buffer with CE") is used as output clock drive and one <code>clr_n</code> pin is available for one MBUFGCE. Which <code>clr_n</code> pin is available depends on the tree clock and leaf clocks of that MBUFGCE. For example: Two clock outs are present and <code>clk_out1</code> = 200 MHz and <code>clk_out2</code> = 100 MHz and drives selected as buffer for both the clocks. And clock grouping is same for both the clocks as both the output frequencies are integer dividers to each other which can be inferred from MBUFGCE and here, <code>clk_out1</code> is tree and <code>clk_out2</code> is leaf clock derived from it. So, <code>clk1_clr_n</code> will be available in this case. This is active-Low signal, and you should tie to 1 for MBUFGCE to be active.

Table 2: Clocking Wizard Output Clock Ports (cont'd)

Port Name	I/O	Description
clk[1-7]_ce	Input	Clock CE: Clock CE pin. Available when the buffer (auto buffer, which means "Buffer with CE") is used as output clock drive, and one CE pin is available for one MBUFGCE. Which CE pin is available depends on the tree clock and leaf clocks of that MBUFGCE. For example, two clock outs are present and clk_out1 = 200 MHz and clk_out2 = 100 MHz and drives selected as "Buffer with CE" for both the clocks. And clock grouping is the same for both the clocks as both the output frequencies are integer dividers, which can be inferred from MBUFGCE, and here, clk_out1 is a tree, and clk_out2 is leaf clock derived from it. So, clk1_ce will be available in this case. This is an active-High signal, and you should tie to 1 for MBUFGCE to be active.

Dynamic Phase Shift Ports

Table 3: Clocking Wizard Dynamic Phase Shift Ports

Port Name	I/O	Description
psclk	Input	Dynamic Phase Shift Clock: Clock for use in dynamic phase shifting
psen	Input	Dynamic Phase Shift Enable: Starts a dynamic phase shift transaction
psincdec	Input	Dynamic Phase Shift increment/decrement: When '1', increments the phase shift of the output clock; when '0', decrements the phase shift
psdone	Output	Dynamic Phase Shift Done: Completes a dynamic phase shift transaction

Status and Control Ports

Table 4: Clocking Wizard Status and Control Ports

Port Name	I/O	Description
reset/resetn	Input	Reset (active- High)/Resetn (active-Low): When asserted, asynchronously clears the internal state of the primitive, and causes the primitive to re-initiate the locking sequence when released
power_down	Input	Power Down: When asserted, places the clocking primitive into low power state, which stops the output clocks
input_clk_stopped	Output	Input Clock Stopped: When asserted, indicates that the selected input clock is no longer toggling
locked	Output	Locked: When asserted, indicates that the output clocks are stable and usable by downstream circuitry
locked_fb	Output	Locked_fb: When asserted, indicates that the feedback clock is stable.
s_axi_*	Input/Output	See Appendix A of the <i>Vivado Design Suite: AXI Reference Guide</i> (UG1037). This signal is available only if dynamic reconfiguration is enabled.
locked_deskew1	Output	Locked_DESKEW1: When asserted, indicates that the DESKEW PD1 Network clocks are stable.

Table 4: Clocking Wizard Status and Control Ports (cont'd)

Port Name	I/O	Description
locked_deskew2	Output	Locked_DESKEW2: When asserted, indicates that the DESKEW PD2 Network clocks are stable.
clkfb_stopped	Output	CLKFBStopped: It is asserted when the feedback clock is lost.
interrupt	Output	Interrupt: This port is available when clock monitor is selected. This port gives the interrupts of the clock monitor feature
clk_oor [3:0]	Output	clk_OOR: This port is available when clock monitor is selected. The bits for this port are high when input clock frequency is out of range than expected. • Bit 0 - User clock 0 • Bit 1 - User clock 1 • Bit 2 - User clock 2 • Bit 3 - User clock 3
clk_glitch [3:0]	Output	clk_glitch: This port is available when clock monitor is selected. The bits for this port are high when there is a glitch in the input clock. • Bit 0 - User clock 0 • Bit 1 - User clock 1 • Bit 2 - User clock 2 • Bit 3 - User clock 3
clk_stop [3:0]	Output	clk_stop: This port is available when clock monitor is selected. The bits for this port are high when clock is stopped on the respective user clock. clock is stopped • Bit 0 - User clock 0 • Bit 1 - User clock 1 • Bit 2 - User clock 2 • Bit 3 - User clock 3

Note: Exposure of every status and control port is individually selectable. To enable interrupt register, you must write to the interrupt enable register with all 1's. This allows you to enable interrupt for clock stop, clock overrun, and clock underrun bits in the interrupt status register.

Register Space

The following sections describe the set of registers applicable when the Dynamic Reconfiguration Mode is selected. All registers are accessed as 32-bit. The tables mention the default value of the registers and a brief description about them.

Clock Configuration Registers

Following table describes clock configuration registers:

Table 5: Clock Configuration Registers

Base Address + Offset (hex)	Register Name	Reset Value (hex)	Access Type	Description
C_BASEADDR + 0x00	Software Reset Register (SRR)	N/A	W	To activate software reset, the value 0x0000_000A must be written to the register. Any other access, read or write, has undefined results.
C_BASEADDR + 0x04	Status Register (SR)	0x00000000	R	Bit[0] = Locked when 1 MMCM/PLL is locked and ready for reconfiguration. The status of this bit is 0 during reconfiguration.
C_BASEADDR + 0x08	Clock Monitor Error Status Register	0x00000000	R	This register gives the error status bits of the clock monitor feature.
C_BASEADDR + 0x0C	Interrupt Status	0x00000000	R/W	Interrupt status for clock stop, clock overrun, and clock underrun. These bits are gated by interrupt enable bits. Interrupts corresponding to the enabled bits in interrupt enable register would be updated in the register.
C_BASEADDR + 0x10	Interrupt Enable	0x00000000	R/W	Interrupt enable for clock stop, clock overrun and clock underrun bits in the interrupt status register.
C_BASEADDR + 0x014	Clock Configuration Register	0000	R/W	Bit[0] = LOAD / SEN: Loads Clock Configuration Register values to the internal register used for dynamic reconfiguration and initiates reconfiguration state machine. This bit should be asserted when the required settings are already written into Clock Configuration Registers. This bit retains to 0 when the dynamic reconfiguration is done and the clock is locked. Bit[1] = SADDR: When written 0, default configuration done in the Clocking Wizard GUI is loaded for dynamic reconfiguration. When written 1, setting provided in the Clock Configuration Registers are used for dynamic reconfiguration.
C_BASEADDR + 0x330	CLKFBOUT_1	TBD	R/W	Note 1
C_BASEADDR + 0x334	CLKFBOUT_2	TBD	R/W	Note 1
C_BASEADDR + 0x338	CLKOUT0_1	TBD	R/W	Note 1
C_BASEADDR + 0x33C	CLKOUT0_2	TBD	R/W	Note 1
C_BASEADDR + 0x340	CLKOUT1_1	TBD	R/W	Note 1
C_BASEADDR + 0x344	CLKOUT1_2	TBD	R/W	Note 1
C_BASEADDR + 0x348	CLKOUT2_1	TBD	R/W	Note 1
C_BASEADDR + 0x34C	CLKOUT2_2	TBD	R/W	Note 1
C_BASEADDR + 0x350	CLKOUT3_1	TBD	R/W	Note 1
C_BASEADDR + 0x354	CLKOUT3_2	TBD	R/W	Note 1
C_BASEADDR + 0x378	CP	TBD	R/W	Note 1
C_BASEADDR + 0x37C	DESKEW_1	TBD	R/W	Note 1
C_BASEADDR + 0x380	DESKEW_2	TBD	R/W	Note 1
C_BASEADDR + 0x384	DIVCLK	TBD	R/W	Note 1
C_BASEADDR + 0x390	INTERPOL_1	TBD	R/W	Note 1

Table 5: Clock Configuration Registers (cont'd)

Base Address + Offset (hex)	Register Name	Reset Value (hex)	Access Type	Description
C_BASEADDR + 0x394	INTERPOL_2	TBD	R/W	Note 1
C_BASEADDR + 0x398	LF	TBD	R/W	Note 1
C_BASEADDR + 0x39C	LOCK_1	TBD	R/W	Note 1
C_BASEADDR + 0x3A0	LOCK_2	TBD	R/W	Note 1
C_BASEADDR + 0x3A8	RES_1	TBD	R/W	Note 1
C_BASEADDR + 0x3AC	RES_2	TBD	R/W	Note 1
C_BASEADDR + 0x3C0	UNLOCK_CNT	TBD	R/W	Note 1
C_BASEADDR + 0x3CC	CLKOUT4_1	TBD	R/W	Note 1
C_BASEADDR + 0x3D0	CLKOUT4_2	TBD	R/W	Note 1
C_BASEADDR + 0x3D4	CLKOUT5_1	TBD	R/W	Note 1
C_BASEADDR + 0x3D8	CLKOUT5_2	TBD	R/W	Note 1
C_BASEADDR + 0x3DC	CLKOUT6_1	TBD	R/W	Note 1
C_BASEADDR + 0x3E0	CLKOUT6_2	TBD	R/W	Note 1
C_BASEADDR + 0x3E4	INTERPOL_3	TBD	R/W	Note 1
C_BASEADDR + 0x3E8	INTERPOL_4	TBD	R/W	Note 1
C_BASEADDR + 0x3F0	CLKFBOUT_3	TBD	R/W	Note 1
C_BASEADDR + 0x3F8	SS	TBD	R/W	Note 1
C_BASEADDR + 0x3FC	CLKFBOUT_4	TBD	R/W	Note 1
C_BASEADDR + 0x3FF to C_BASEADDR + 0xFFF	Undefined	Undefined	N/A	Do not read/write these registers.

Notes:

1. Refer to [Dynamic Reconfiguration through AXI4-Lite](#) for more details on these registers.

Designing with the Core

This chapter includes guidelines and additional information to make designing with the core easier.

General Design Guidelines

- Provide the available input clock information for frequency and jitter.
- If the same input clock is used by another logic, and is also the output of the global buffer, then provide the no buffer or global buffer option for the source type. If the input clock is used only by the core, then provide a clock-capable pin as the source type.

Clocking

Up to seven output clocks with different frequencies can be generated for the required circuitry.

Resets

- The Clocking Wizard has an active-High asynchronous reset signal for the clocking primitive.
- The core must be held in `reset` during clock switch over.
- When the input clock or the feedback clock is lost, the `input_clk_stopped` or `clkfb_stopped` status signal is asserted. After the clock returns, the `input_clk_stopped` signal is de-asserted and a `reset` must be applied.

Functional Overview

The Clocking Wizard is an interactive graphical user interface (GUI) that creates a clocking network based on design-specific needs. The required clock network parameters are organized in a linear sequence, so that you can select only the desired parameters. Advanced users can use the wizard to explicitly configure their chosen clocking primitive while beginners can let the wizard automatically determine the optimal primitive and configuration based on the features required for their individual clocking networks.

If you are already familiar with the digital clock manager (DCM) and phase-locked loop (PLL) wizards, see [Appendix A: Migrating](#) for information on usage differences.

Clocking Features

Major clocking-related functional features desired and specified can be used by the wizard to select an appropriate primitive. Incompatible features are automatically dimmed out to help the designer evaluate feature trade-offs. Clocking features include:

- Phase alignment
- Spread spectrum
- Minimization of output jitter
- Allowance of larger input jitter
- Minimization of power
- Dynamic phase shift
- Dynamic reconfiguration
- Auto Primitive
- Clock Monitor
- Clock Grouping
- Auto Buffer
- Deskew connection logic
- Safe Clock Startup
- MBUFGCE support

Input Clocks

One input clock is the default behavior, but you can select a secondary clock source. Only the timing parameters of the input clocks in their specified units is required; the wizard uses these parameters as needed to configure the output clocks.

Input Clock Jitter Option

The wizard allows you to specify the input clock jitter either in UI or PS units using a drop-down menu.

Output Clocks

The number of output clocks is user-configurable. The maximum number allowed depends upon the selected device or primitive and the interaction of the major clocking features you specify. For MMCM, a maximum of seven, and for XPLL and DPLL, a maximum of four output clocks can be configured. If the primitive selected is Auto, all the seven output clocks can be configured. Input the desired timing parameters (frequency, phase, and duty cycle) and let the Clocking Wizard select and configure the clocking primitive and network automatically to comply with the requested characteristics. If it is not possible to comply exactly with the requested parameter settings due to the number of available input clocks, best-attempt settings are provided. When this is the case, the clocks are ordered so that `clk_out1` is the highest-priority clock and is most likely to comply with the requested timing parameters. The wizard prompts you to set the frequency parameter settings before setting the phase and duty cycle settings.



TIP: The port names in the generated circuit can differ from the port names used on the original primitive.

Clock Buffering and Feedback

In addition to configuring the clocking primitive within the device, the wizard also assists with constructing the clocking network. Buffering options are provided for both input and output clocks. Feedback for the primitive can be user-controlled or left to the wizard. If automatic feedback is selected, the feedback path is matched to timing for `clk_out1`.

Optional Ports

All primitive ports are available for user-configuration. You can expose any of the ports on the clocking primitive, and these are provided in the source code.

Primitive Override

Most of the configuration parameters of the primitive are also user-configurable. You can override the parameter values with other values if the values of your choice are in the valid range and suit the clocking solutions.

Note: This feature is not available when the primitive type is Auto. If you want to override the primitive attributes, do not select Auto.

Clock Monitor

The Clock Monitor feature allows you to monitor the clocks in a system, typically inputs to MMCM/PLL. This detects the change in frequency of the clock, glitch in the clock or a clock stop. It also gives the option to provide the tolerance required. Depending on the tolerance given by you the Clock Monitor function varies. For example, if you want to see an error only if the frequency is 1 MHz higher than the requested frequency, then set the tolerance as 1 MHz.

Clock Stop

The Clock Stop goes high when the clock is flat-lined for more than 10 clock cycles. The clock is stale for 10 or more clock cycles, it initiates the calculation to detect the stop and signals the `clk_stop` high. The `clk_stop` signal will not become high immediately and can take up to 256 clock cycles from the time the clock is flat. This calculation requires a set of reference clock cycle periods, time at which the `clk_stop` go high depends on the ratio of `user_clk` and `ref_clk`.

Clock Glitch

The Clock Monitor can detect the glitch in the user clock. The minimum glitch it can detect in the user clock is one clock period of the reference clock. The Clock Glitch condition might overlap with the Clock Overrun.

Clock Out of Range

The Clock Monitor detects if the user clock frequency exceeds or goes down below the required frequency.

Note: Clock under run signal may also go high during the stop condition, if the frequency of the signals goes much lower than the desired frequency.

Note: There might be cases where OOR/Glitch is not detected if the difference between maximum user clock frequency (Ex: 280 MHz) to minimum user clock frequency (Ex: 10 MHz) is huge.

Safe Clock Startup

This feature enables a stable and valid clock at the output using BUFGCE after Locked is sampled High for eight input clocks. The sequencing feature enables clocks in a sequence according to the number entered in the GUI corresponding to each output clock. The delay between two enabled output clocks in a sequence is eight cycles of the second clock in the sequence clock.

This feature is useful for a system where modules need to start operating one after the other. When the Safe Clock Startup option is selected, only BUFGCE is allowed as driver for all the output clocks, and there is no choice to switch back to other buffer.

MBUFGCE Enhancements

When MBUFGCE is selected explicitly, or IP Inferred MBUFGCE, a new Parameter `CE_SYNC_EXT` will be available to the user. MBUFGCE Inference by IP happens on selecting either "Buffer" or "Buffer with CE." There are MODEL PARAMETERS for getting information on how many MBUFGCE are inferred (`C_AUTO_NUMMBUFGCE`) and which clocks are connected through MBUFGCE (`C_CLK_TREE<1-7>`). The user will get all this information from the summary table of GUI.

`CLRB_LEAF` pin on MBUFGCE is active low asynchronous clear, used to reset BUFDIV_LEAF clock dividers of MBUFGCE.

The optimal circuit will help synchronize `CE` and `CLRB_LEAF` pins of MBUFGCE to LOCKED and Reset signals, guaranteeing the correct startup behavior of the MBUFGCE Clock outputs.

You can select "CE and CLR SYNC EXTERNAL TO CORE" from Output Clocks Tab to have the circuitry outside the IP. The default value of `CE_SYNC_EXT` is false. This means an optimal circuit needs to be implemented by IP Core inside the wizard.

The user needs to set the parameter to true if the user wants to implement a synchronized structure outside IP. In that case, IP will directly get those signals from outside and will feed to respective signals of `CE` and `CLRB_LEAF` signals of MBUFGCE.

Note: For more details please refer:

- Versal ACAP Hardware, IP, and Platform Development Methodology Guide ([UG1387](#))
- Versal ACAP Clocking Resources Architecture Manual ([AM003](#))

Auto Primitive

This feature helps instantiating the clocking primitive that best fits your requirements with minimum usage of clocking resources, high performance, and better clock routing. All clocking features and optional ports would be in unselected state when you select primitive as Auto. You need to exclusively enable the options that are required. The following table explains the selection criteria depending on the clocking features that you select.

Table 6: Auto Primitives

Feature	Auto Primitive Selection	Selection Criteria
Phase Alignment	MMCM	This feature is only supported in MMCM
Dynamic Phase Shift	MMCM or DPLL or MBUFGCE	The selection criteria is: - If number of output clocks are > 4, MMCM is Auto Primitive - If the number of output clocks ≤ 4 and the Selected Output Clocks ≤ 500 MHz and 50% Duty Cycle, DPLL is Auto Primitive or else it is MMCM - If number of output clocks are ≤ 4 and any of the clocks PI Control is selected as DESKEW_PD2, then MMCM is the Auto Primitive. - If number of output clocks are ≤ 4 and none of the clocks PI Control selected as DESKEW_PD1 or DESKEW_PD2 and drives is selected as buffer and phase is 0.00 for all the clocks and duty cycle is 50 and clock grouping is auto for all the clocks. And all the used output clocks are integer divides (/1, /2, /4 or /8) to each other then MBUFGCE is Auto Primitive.
Secondary Input Clock	MMCM	This feature is only supported in MMCM

Table 6: Auto Primitives (cont'd)

Feature	Auto Primitive Selection	Selection Criteria
Number Output Clocks selected > 4	MMCM or DPLL or MBUFGCE	The selection criteria is: - If the number of output clocks are > 4 then MMCM is set to Auto Primitive - If the number of output clocks ≤ 4 and the selected output clocks ≤ 500 MHz at 50% Duty Cycle, the DPLL is set to Auto Primitive or else it is set to MMCM - If number of output clocks are ≤ 4 and any of the clocks DYN_PS is selected as DESKEW_PD2, then MMCM is the Auto Primitive - If number of output clocks are ≤ 4 and none of the clocks PI Control selected as DESKEW_PD1 or DESKEW_PD2 and drives selected as buffer and phase is 0.00 for all the clocks. Duty Cycle is 50 and Clock Grouping is Auto for all the clocks. All the used output clocks are integer divides (/1, /2, /4 or /8) to each other then MBUFGCE is Auto Primitive.
Safe Clock Startup and Safe Clock Mode as Deskew Mode	MMCM	DESKEW Mode is not supported for DPLL, as <code>clk_in_deskew</code> connection must not come from <code>clk_out</code> path
Non zero Phase	MMCM or DPLL	The selection criteria is: - If the number of output clocks are > 4 then MMCM is set to Auto Primitive - If the number of output clocks ≤ 4 and the selected output clocks ≤ 500 MHz at 50% Duty Cycle, the DPLL is set to Auto Primitive or else it is set to MMCM
Input Clock Stopped (port)	MMCM	This port is available only in MMCM
Clkfb_stopped (port)	MMCM	This port is available only in MMCM

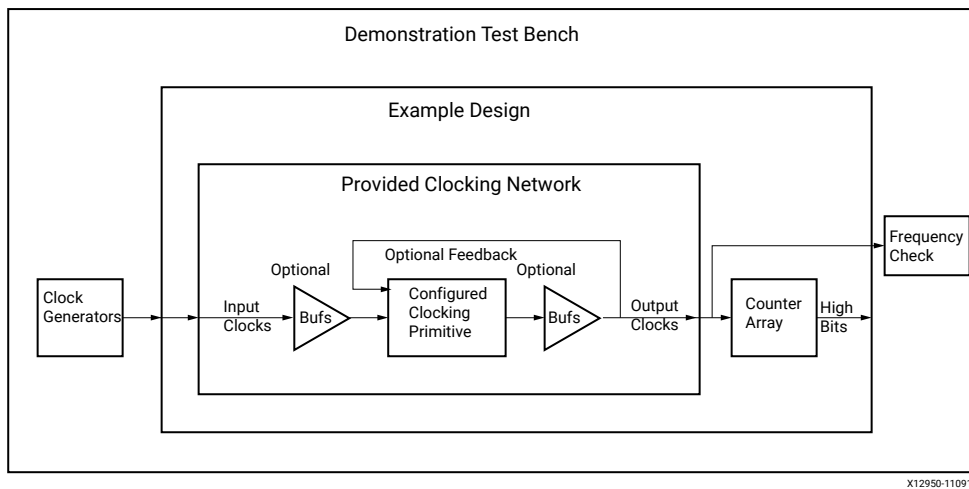
Table 6: Auto Primitives (cont'd)

Feature	Auto Primitive Selection	Selection Criteria
Reset, Locked, Power_down Port Input Frequency Ranges, Output Frequency ranges, and VCO Frequency Ranges Note: The range values will vary depending on the part/board/speed grade selected. Refer to <i>Versal AI Core Series Data Sheet: DC and AC Switching Characteristics (DS957)</i> for DC and AC switching characteristics.	MMCM OR DPLL	MMCM as AUTO_PRIMITIVE is chosen when: - If the number of output clocks are > 4 then MMCM is set to Auto. If the number of output clocks < 4 with random duty cycle and phase - If the number of output clocks < 4 and clocking software algorithm is not able to find the solution - If number of output clocks are <=4 and any of the clocks DYN_PS is selected as DESKEW_PD2, then MMCM is the Auto Primitive - If the number of output clocks ≤ 4 with random duty cycle and phase and the selected output clocks ≤ 500 MHz at 50% Duty Cycle DPLL as AUTO_PRIMITIVE is chosen when: Note: Reset and power down are mapped to the same port. For backward compatibility, reset is shown as a port.
Output Clocks <= 4	DPLL	If Duty Cycle of all clocks = 50%
	MMCM	If any clock Duty Cycle != 50%
	MBUFGCE	If Duty Cycle of all clocks = 50% and phase of all clocks = 0 and clock grouping of all clocks is Auto and all the output clock frequencies are integer divides to each other (/1, /2, /4 and /8) For example: Four clock outs are present with above criteria and having frequencies as ck_out1 = 200 MHz, clk_out2 = 100 MHz, clk_out3 = 50 MHz and ck_out4 = 25 MHz. In this case MBUFGCE is inferred as Auto Primitive as the clocks are divides to each other
ZHOLD	DPLL	The selection criteria is: If the number of output clocks ≤ 4 with ZHOLD is enabled and the selected output clocks ≤ 500 MHz at 50% Duty Cycle, then the DPLL is set to Auto Primitive

Design Environment

The following figure shows the design environment provided by the wizard to assist in integrating the generated clocking network into a design. The wizard provides a synthesizable and downloadable example design to demonstrate how to use the network and allows you to place a very simple clocking network in the device. A sample simulation test bench, which simulates the example design and illustrates output clock waveforms with respect to input clock waveforms, is also provided.

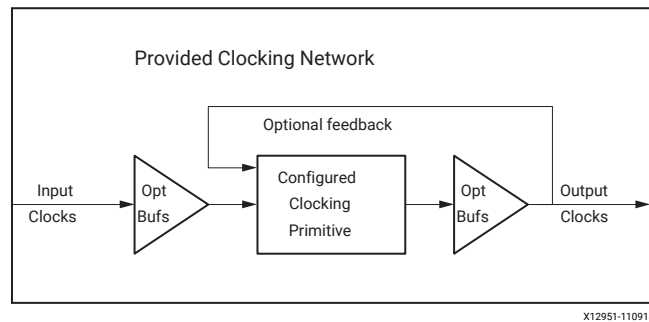
Figure 3: Clocking Network and Support Modules



Core Architecture

The Clocking Wizard generates source code HDL to implement a clocking network. The generated clocking network typically consists of a clocking primitive MMCM, DPLL, XPLL, and some additional circuitry which includes buffers and clock pins. The network is divided into segments as illustrated in the following figure. Details of these segments are described in the following sections.

Figure 4: Provided Clocking Network



Input Clocks

Up to two input clocks are available for the clocking network. Buffers are optionally inserted on the input clock paths based on the buffer type that is selected.

Primitive Instantiation

The primitive, either user or wizard selected, is instantiated into the network. Parameters on primitives are set by the wizard, and can be overridden by the users. Unused input ports are tied to the appropriate values. Unused output ports are labeled as such.

Feedback

If phase alignment is not selected, the feedback output port on the primitive is automatically tied to the feedback input port. If phase alignment with automatic feedback is selected, the connection is made, but the path delay is matched to that of `clk_out1`. If user-controlled feedback is selected, the feedback ports are exposed.

Output Clocks

Buffers that are user-selected are added to the output clock path, and these clocks are provided.

I/O Signals

All ports are optional, with the exception that at least one input and one output clock are required. Availability of ports is controlled by user-selected parameters. Any port that is not exposed is either tied off or connected to a signal labeled unused in the delivered source code.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard Vivado® design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

If you are customizing and generating the core in the Vivado IP Integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP Integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, you can run the `validate_bd_design` command in the Tcl console.

Note: Figures in this chapter are illustrations of the Vivado IDE. This layout might vary from the current version.

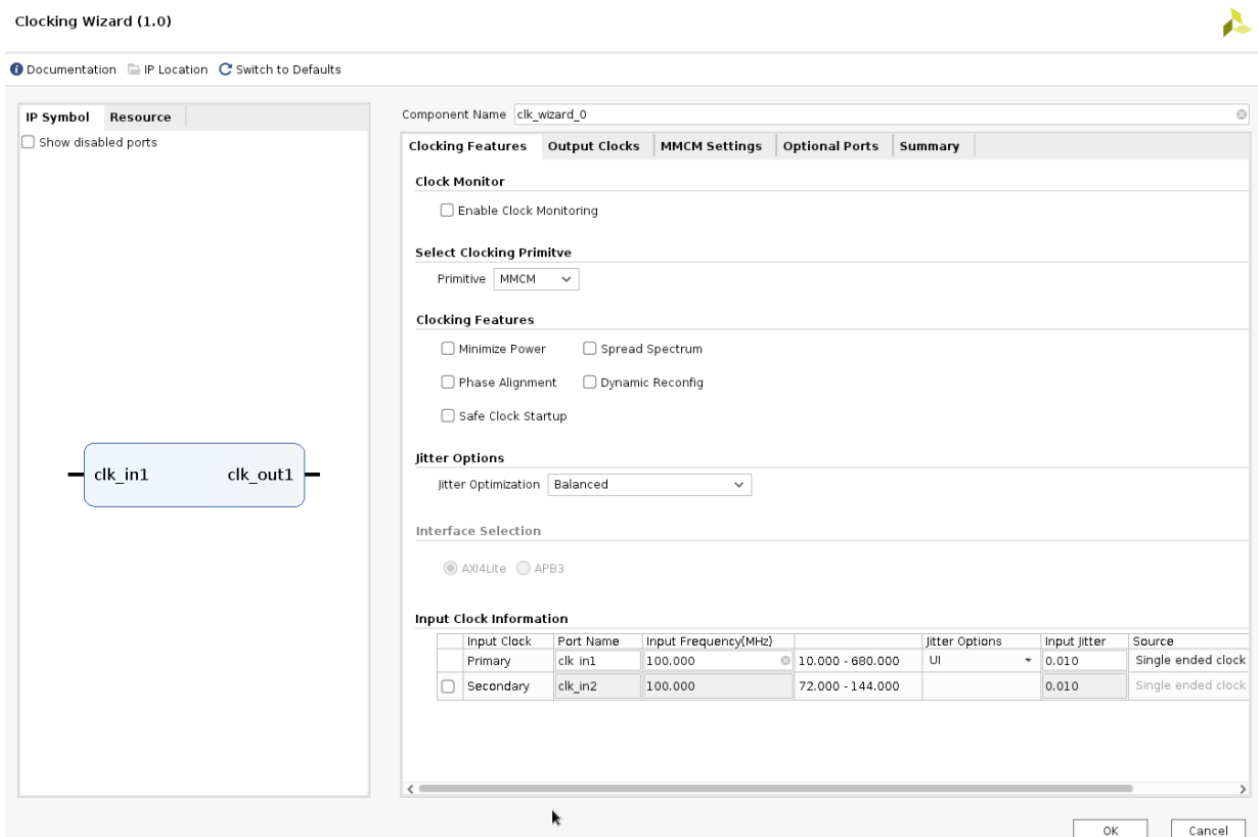
Clock Manager Type (Primitive Selection)

Versal® devices have MMCME5, XPLL, and DPLL clocking primitives. You have the option to configure either of these by selecting the primitive. Depending on the primitive that you select, some features are either enabled or disabled.

Clocking Features

The first tab of the GUI allows you to identify the required features of the clocking network and configure the input clocks.

Figure 5: Clocking Features



Selecting Clocking Features

The available clocking features are shown for the selected target device. You can select as many features as desired; however, some features consume additional resources, and some can result in increased power consumption. Additionally, certain combinations of features are not allowed. When using IP Integrator, frequency, phase, and clock domain properties of the output clocks are automatically propagated and any change in input clock properties reflect on all the outputs.

Note: The Port Renaming tab has been removed from the Clocking Wizard Interface as the IP does not support the renaming of ports in the IP Integrator.

Configuring Input Clocks

The above options are unselected in Auto Primitive selection. There are two input clocks available and depending on your selection, the reference clock can be switched from one to another. The GUI provides an option to select the secondary input clock to enable the additional input clock.

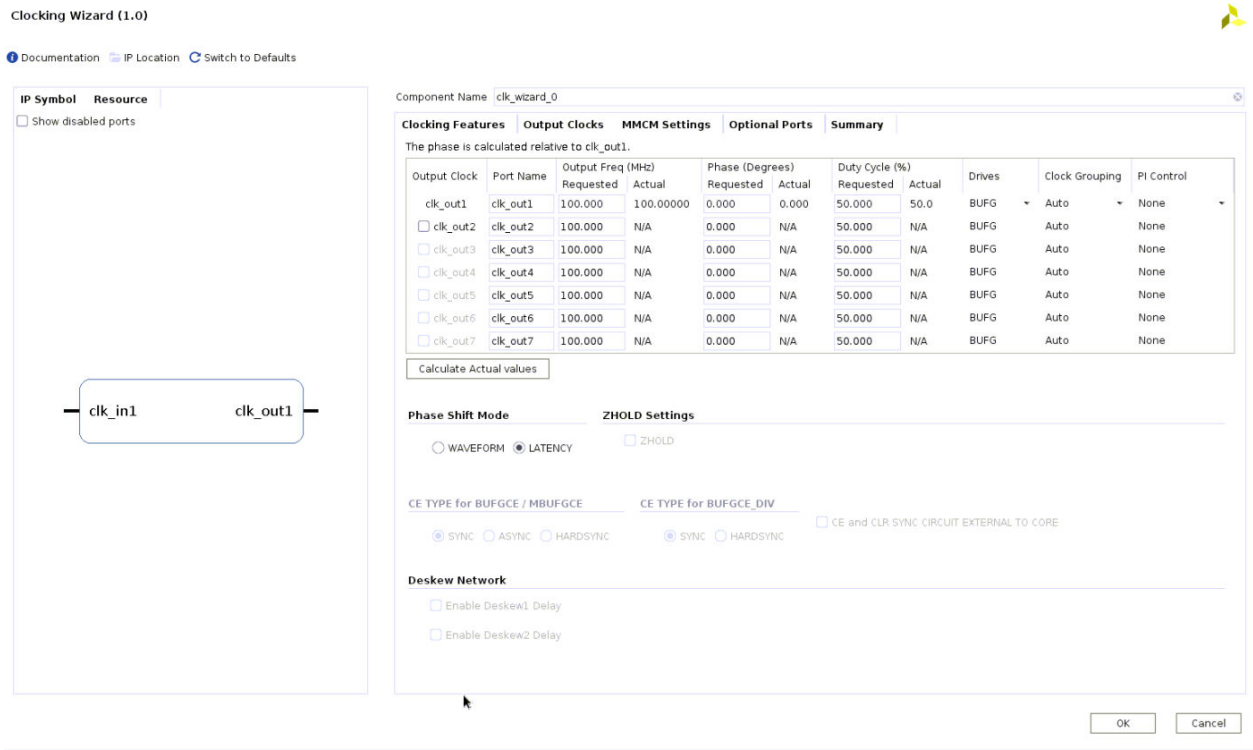
You can select the buffer type that drives the input clock, and this is then instantiated in the provided source code. If the input buffers are located externally, selecting No Buffer leaves the connection blank. If Phase Alignment is selected, you will not have access to pins that are not dedicated clock pins because the skew introduced by a non-clock pin is not matched by the primitive. You can choose the units for input clock jitter by selecting either the UI or PS drop-down menu. The input jitter box accepts values based on this selection.

In the IP Integrator, when the Clocking Wizard IP is selected to target a board part, the frequency values that are generated to the primary and secondary clocks are displayed in a floating number format. For example, if the primary clock frequency is 100 MHz, then it displays as 100.000 instead of 100.

Output Clock Settings

The second page of the GUI (the following figure) configures requirements for the output clocks. Each selected output clock can be configured on this screen.

Figure 6: Output Clocks



Configuring Output Clocks

To enable an output clock, click on the box located next to it. Output clocks must be enabled sequentially. You can now rename the output clocks in the output clock table itself.

You can specify values for the output clock frequency, phase shift, and duty cycle assuming that the primary input clock is the active input clock. You can select drives, clock grouping and PI control for each output clock. The Clocking Wizard attempts to derive a clocking network that meets your criterion exactly. In the event that a solution cannot be found, best attempt values are provided and are shown in the actual value column. Actual frequencies are calculated to limit the values to five decimal places. Achieving the specified output frequency takes precedence over implementing the specified phase, and phase takes higher precedence over duty cycle in the clock network derivation process. The precedence of deriving the circuits for the `clk_out` signals is `clk_out1 > clk_out2 > clk_out3` and so on. Therefore, finding a solution for `clk_out1` frequency has a higher priority. Values are recalculated every time an input changes. Because of this, it is best to enter the requirements from top to bottom and left to right. This helps to pinpoint requested values that cannot be supported exactly. If phase alignment is selected, the phase shift is with respect to the active input clock.

Note: In the Versal ACAP Clocking Wizard, there are no individual parameters for each clock. The parameters must be provided as a string for all the output clocks when using batch mode. For example, if you want to feed the requested clock frequencies for the clocks, the parameter CLKOUT_REQUESTED_OUT_FREQ must be assigned a value of 120, 100, 300, 450, 200, 100, and 380. It is the combination of all output frequencies from 1 to 7. If the string does not contain 7 values, it fills the initial frequencies and takes the default values for the rest. The entire table in "Output Clocks" Tab is a single widget, and the SW takes the information. The user must either click "Calculate Actual Values" button below the table or move the cursor outside the widget. This is a JAVA Widget restriction, which intern helps the processing to be fast.

Note: For BUFGCE_DIV, DIVIDER value is set to 1. To use other values, select "No_buffer" and generate IP. Use Utility_buffer to select BUFGCE_DIV and pass the needed DIV value.

Note: For BUFG_GT, DIVIDER value is set to 1.

Reset Type

You can select Reset Type as active-High or active-Low when RESET is enabled. Default value is active-High.



RECOMMENDED: Xilinx recommends using the active-High reset in the design.

Choosing Feedback

Feedback selection is only available when phase alignment is selected. When phase alignment is not selected, the output feedback is directly connected to the input feedback. For designs with phase alignment, choose automatic control on-chip if you want the feedback path to match the insertion delay for CLK_OUT1. You can also select user-controlled feedback if the feedback is in external code. If the path is completely on the FPGA, select on-chip; otherwise, select off-chip. For designs that require external feedback and related I/O logic, choose automatic control off-chip feedback. You can choose either single-ended or differential feedback in this mode. The wizard generates the core logic and logic required to route the feedback signals to the I/O.

Feedback Signaling

When the Phase Alignment feature is enabled, the automatic control on-chip Clkfb_in and Clkfb_out ports are not exposed in the IP.

Figure 7: Automatic Control on-Chip

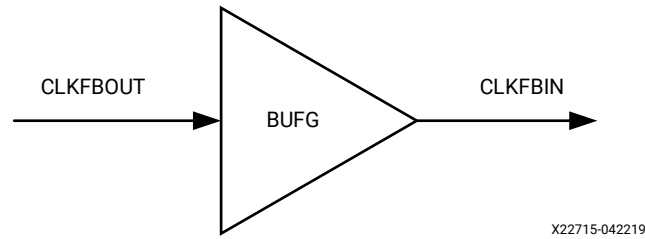


Figure 8: Automatic Control off-Chip

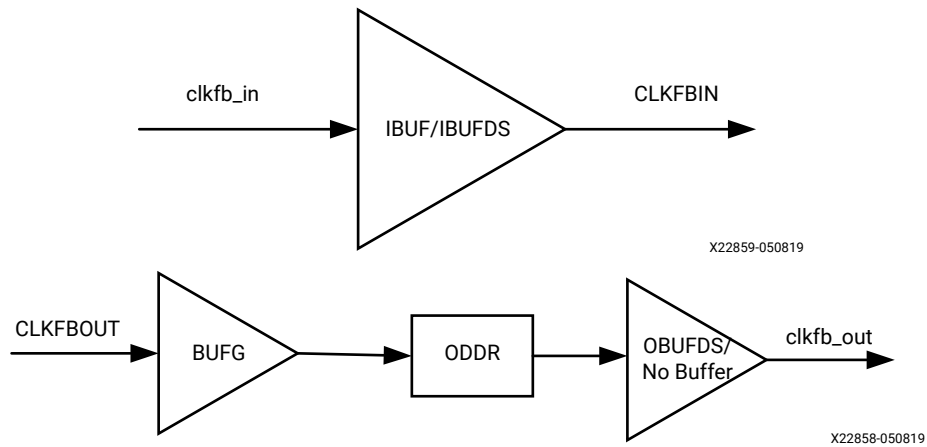
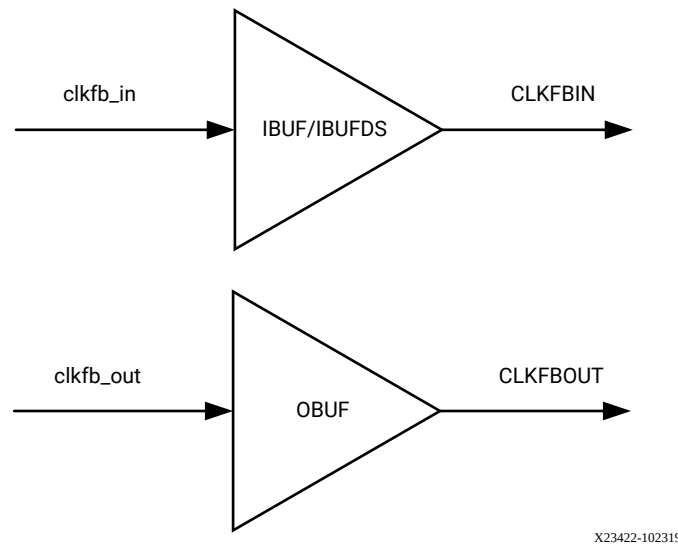


Figure 9: User Controlled ON or OFF Chip



When the Phase Alignment feature is disabled, the Clkfb_in and Clkfb_out ports are not exposed to the IP. CLKFBIN is directly connected to CLKFBOUT.

Phase Shift Mode

Select whether the phase-shifted clock should be modeled into the clock waveform or latency. No multicycle constraint is needed when modeled through latency. The PHASE_SHIFT_MODE property is set in the generated XDC. For more information, see *Vivado Design Suite User Guide: Design Analysis and Closure Techniques* ([UG906](#)).

For more information on Phase Shift Mode, see *Versal ACAP Clocking Resources Architecture Manual* ([AM003](#)).

Spread Spectrum

When Spread Spectrum (SS) is selected, CLK_OUT3 and CLK_OUT4 are not available. Divide values of these outputs are used for SS modulation frequency generation. There are four modes available for Spread Spectrum Mode:

- DOWN_LOW
- DOWN_HIGH
- CENTER_LOW
- CENTER_HIGH

The available modulation frequency range is 25 – 250 kHz.

Spread Spectrum calculation details are described in the following figures:

Figure 10: Spread Spectrum Mode (Center Spread)

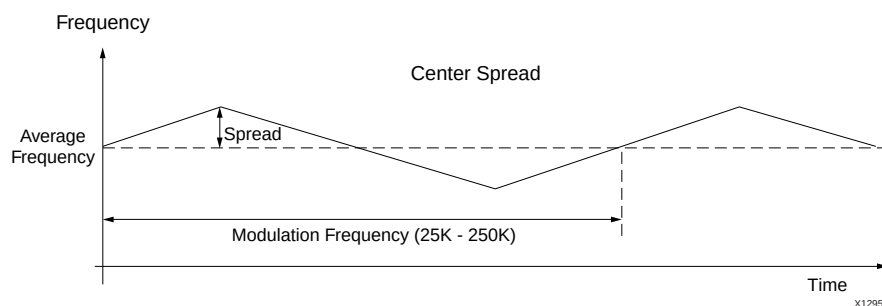
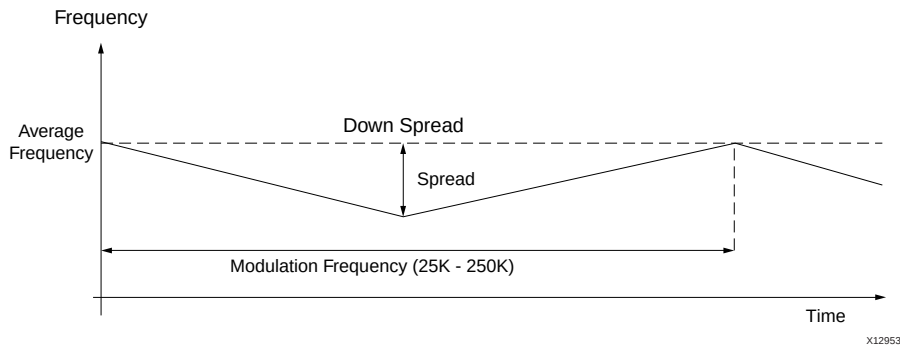


Figure 11: Spread Spectrum Mode (Down Spread)



Note: Input_clock_frequency is in Hz.

For spread:

- If (SS_Mode = CENTER_HIGH):

$$spread (ps) = \pm \left[\frac{D*O}{Input_clock_frequency*(M - 0.125*4)} - \frac{D*O}{Input_clock_frequency*M} \right]$$

- If (SS_Mode = CENTER_LOW):

$$spread (ps) = \pm \left[\frac{D*O}{Input_clock_frequency*(M - 0.125*4)} - \frac{D*O}{Input_clock_frequency*M} \right]$$

- If (SS_Mode = DOWN_HIGH):

$$spread (ps) = + \left[\frac{D*O}{Input_clock_frequency*(M - 0.125*4)} - \frac{D*O}{Input_clock_frequency*M} \right]$$

- If (SS_Mode = DOWN_LOW):

$$spread (ps) = + \left[\frac{D*O}{Input_clock_frequency*(M - 0.125*4)} - \frac{D*O}{Input_clock_frequency*M} \right]$$

In the above equatons, M is CLKFBOUT_MULT_F, D is DIVCLK_DIVIDE, and O is respective CLKOUTX_DIVIDE.

For Modulation Frequency:

- O2 and O3 are calculated by the write_device_image in implementation. Same calculation is done in the wizard to get actual modulation frequency value.
- Then based on what O2 and O3 is calculated, the actual modulation frequency is calculated:

- If (SS_Mode = CENTER_HIGH or SS_Mode = CENTER_LOW)

$$Actual_modulation_frequency (average) = \frac{(Input_clock_frequency*M)}{(O2*O3)*16*D}$$

- If (SS_Mode = DOWN_HIGH)

$$Actual_modulation_frequency (average) = 0.5 * \left[\frac{(Input_clock_frequency*M)}{D*2^3*8} + \frac{Input_clock_frequency*(M - 0.5)}{2^3*D*8} \right]$$

- If (SS_Mode = DOWN_LOW)

$$Actual_modulation_frequency(average) = 0.5 * \left[\frac{Input_clock_frequency * M}{D * 2 * 3 * 8} + \frac{Input_clock_frequency * (M - 0.25)}{2 * 3 * D * 8} \right]$$



IMPORTANT! Actual modulation frequency may deviate within +/- 10% of the requested modulation frequency for some settings.

Safe Clock Startup

This feature enables a stable and valid clock at the output using BUFGCE after Locked is sampled High for eight input clocks.

Following are the options that will be enabled when this option is selected.

Figure 12: Output Clocks with Safe Clock Startup and Clock Sequencing for MMCM

The screenshot shows the Clocking Wizard (1.0) interface. On the left, a block diagram shows a 'reset' input and a 'clk_in1' input connected to a block that outputs 'locked', 'clk_out1', 'clk_out2', 'clk_out3', and 'clk_out4'. The main window displays the 'clk_wizard_0' component configuration. The 'Clocking Features' tab is active, showing a table of output clocks and their settings.

Output Clock	Port Name	Output Freq (MHz)	Requested	Actual	Phase (Degrees)	Requested	Actual	Duty Cycle (%)	Requested	Actual	Drives	Clock Grouping	PI Control
clk_out1	clk_out1	100.000	100.000000	0.000	0.000	50.000	50.0	50.000	50.0	50.0	BUFGCE	Auto	None
☒ clk_out2	clk_out2	100.000	100.000000	0.000	0.000	50.000	50.0	50.000	50.0	50.0	BUFGCE	Auto	None
☒ clk_out3	clk_out3	100.000	100.000000	0.000	0.000	50.000	50.0	50.000	50.0	50.0	BUFGCE	Auto	None
☒ clk_out4	clk_out4	100.000	100.000000	0.000	0.000	50.000	50.0	50.000	50.0	50.0	BUFGCE	Auto	None
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	50.000	N/A	N/A	BUFGCE	Auto	None
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	50.000	N/A	N/A	BUFGCE	Auto	None
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	50.000	N/A	N/A	BUFGCE	Auto	None

Below the table, the 'Use Clock Sequencing' option is checked. The 'SAFELOCK_STARTUP_MODE' is set to 'BUFGCE MODE'. The 'Sequence Number' for all selected clocks is set to 1. The 'Phase Shift Mode' is set to 'LATENCY' and 'ZHOLD' is checked.

You can configure the sequence number from 1 to the maximum number of clocks selected. The Clocking Wizard does not allow any break in the sequence from one to the maximum in the table. The frequency of the output clock in the sequence must not be more than eight times that of the output clock next in sequence.

Reset and Locked Ports are available as soon as this option is selected as the mentioned signals will be used to implement safe clock circuitry.

The safe clock startup circuit will vary depending on the safe clock startup mode you choose.

Safe Clock Startup Mode

This mode selection will determine how the hardware structure should be for selected outputs.

The options for this mode are:

1. DESKEW_MODE
2. BUFGCE_MODE

DESKEW_MODE will help operate the clocking circuit for high frequencies compared to BUFGCE_MODE selection. Hence, DESKEW_MODE is the default value for this parameter.

But, DESKEW_MODE cannot be selected if the user has more than 2 output clocks in MMCM/XPLL (As there are a maximum of two deskew networks present for MMCM/PLL), and this mode is not supported for DPLL because of Deskew connection rules restriction.

Versal offers a new CE_TYPE property, which lets you specify if the CE input to the BUFGCE is timed (CE_TYPE=SYNC) or uses an internal 3-stage synchronizer (CE_TYPE=HARDSYNC). In the case of HARDSYNC, the CE path is not timed (CE timing arc disabled), and the BUFGCE is enabled within 3 or 4 clock cycles. However, there is no guarantee that all the BUFGCE in a clock region that employ the CE_TYPE=HARDSYNC 3-stage synchronizer are enabled within the same number of clock cycles.

Figure 13: Schematic for safe clock startup in Deskew Mode for MMCM

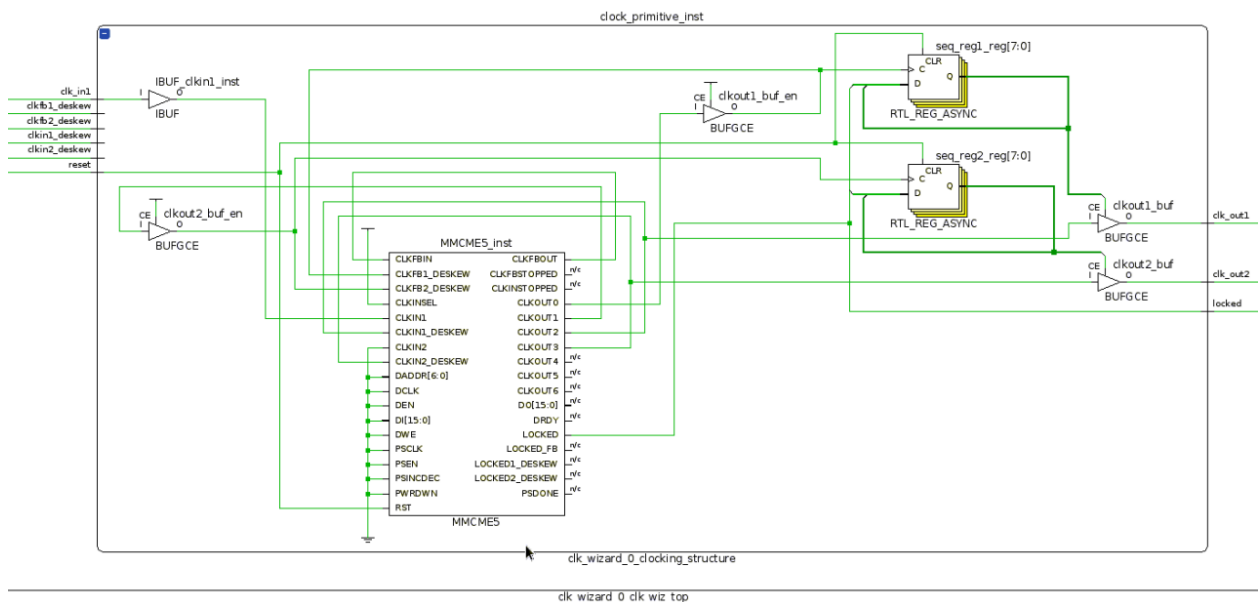


Figure 14: Schematic for safe clock startup in BUFGCE mode

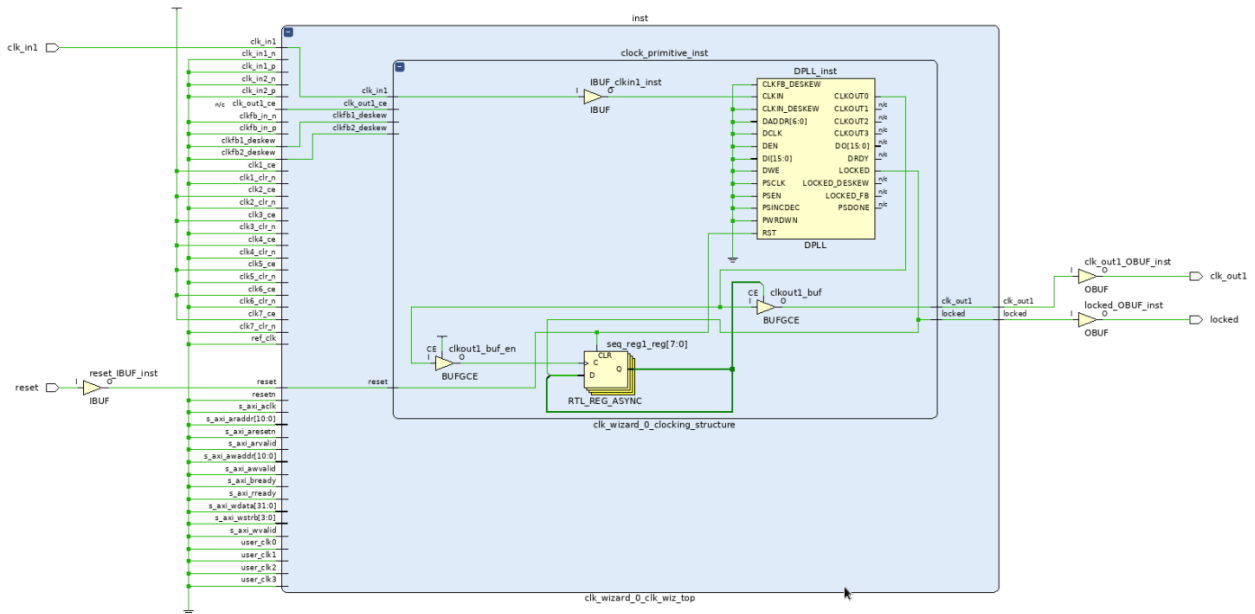
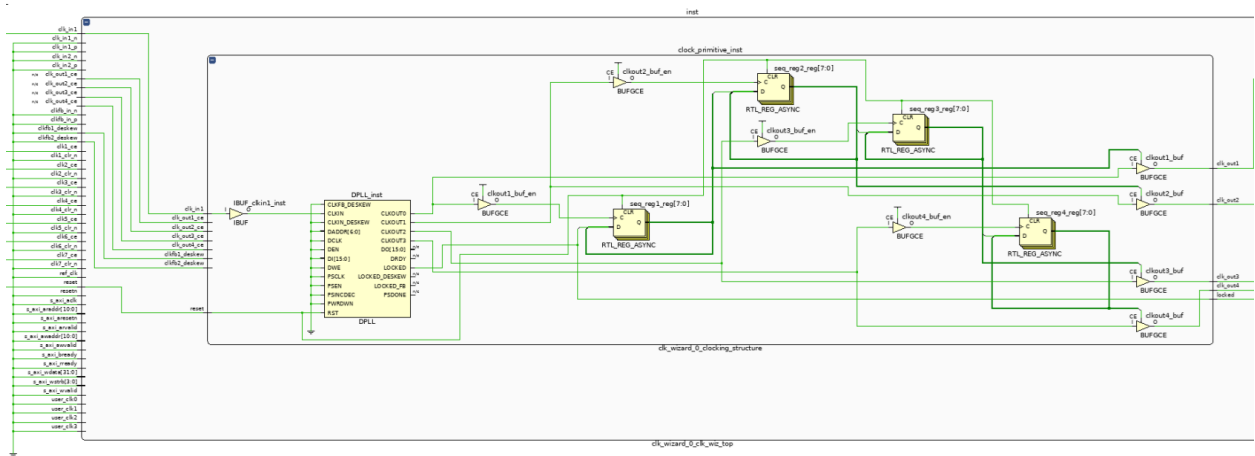


Figure 15: Schematic with the sequencing of four clocks



Note:

- DESKEW_MODE will not be possible in DPLL, as clk_in_deskew connection in DPLL must be from clock input. So, In DPLL, Only BUFGCE Mode will be allowed
- When the user selects DESKEW_MODE, set CE_TYPE to SYNC
- When the user selects BUFGCE_MODE, by default, CE_TYPE is set to HARDSYNC, but the user can change the value to other CE Type.
- In any Mode, Timing is not guaranteed beyond a certain frequency (Frequency will vary for each speed grade)

- In DESKEW_MODE, the user needs to manually modify or adjust the deskew delay to improve the timing to resolve the timing issue.
- For more details, refer *Versal ACAP Hardware, IP, and Platform Development Methodology Guide* ([UG1387](#))

Primitive Overrides

One or more pages of device and primitive specific parameter overrides are displayed.

Overriding Calculated Parameters

The Clocking Wizard selects optimal settings for the parameters of the clocking primitive. You can override any of these calculated parameters as per your requirement. When the Allow Override mode is selected, the overridden values are used as primitive parameters instead of the calculated values. The wizard uses the settings for timing calculations, and any settings changed here are reflected in the summary pages.

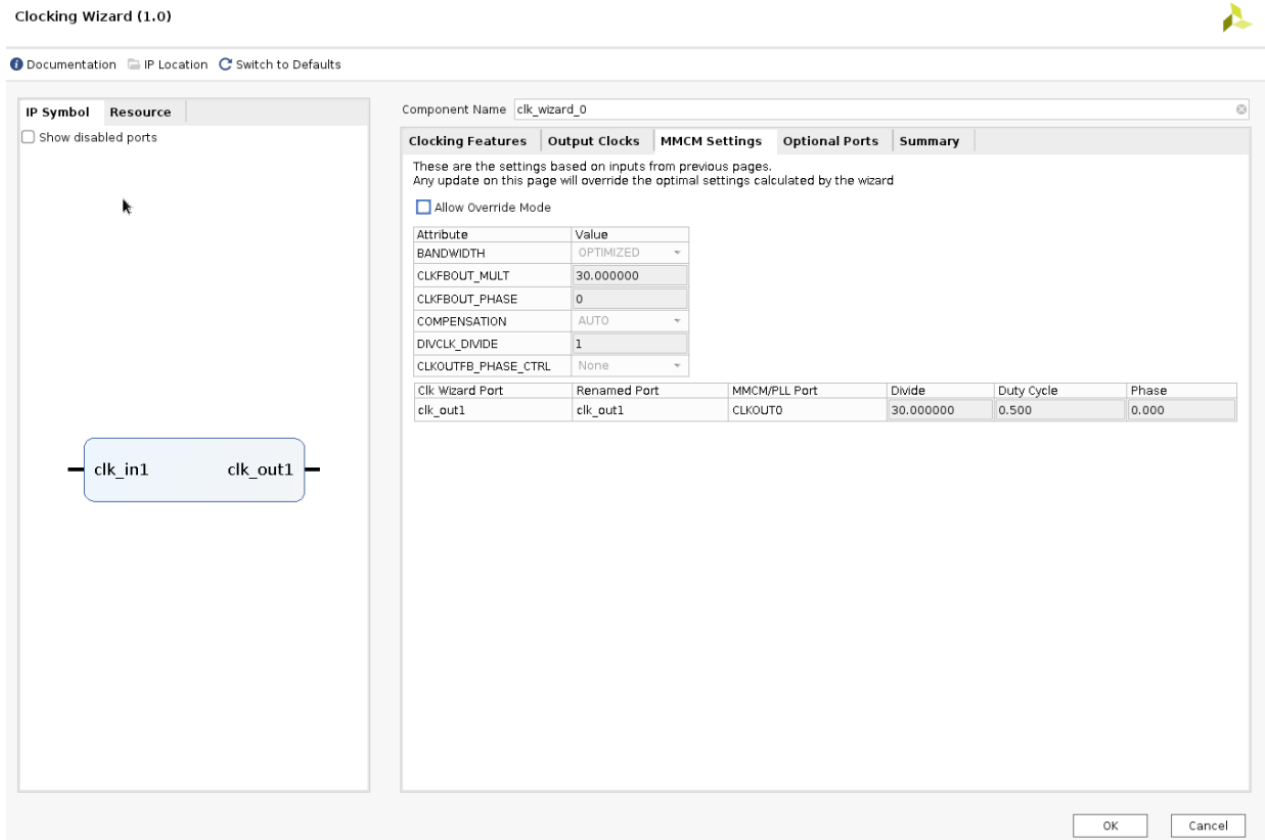


IMPORTANT! *It is important to verify that the values you are choosing to override are correct because the wizard implements what you have chosen even if it causes issues with the generated network.*

Parameters listed are relevant for the physical clocks on the primitive, rather than the logical clocks created in the source code. For example, to modify the settings calculated for the highest priority `CLK_OUT1`, you must modify `CLKOUT0 *` parameters, and not the `CLKOUT1 *` parameters for a MMCM or PLL.

The generated source code contains the input and output clock summaries as shown in [Summary](#).

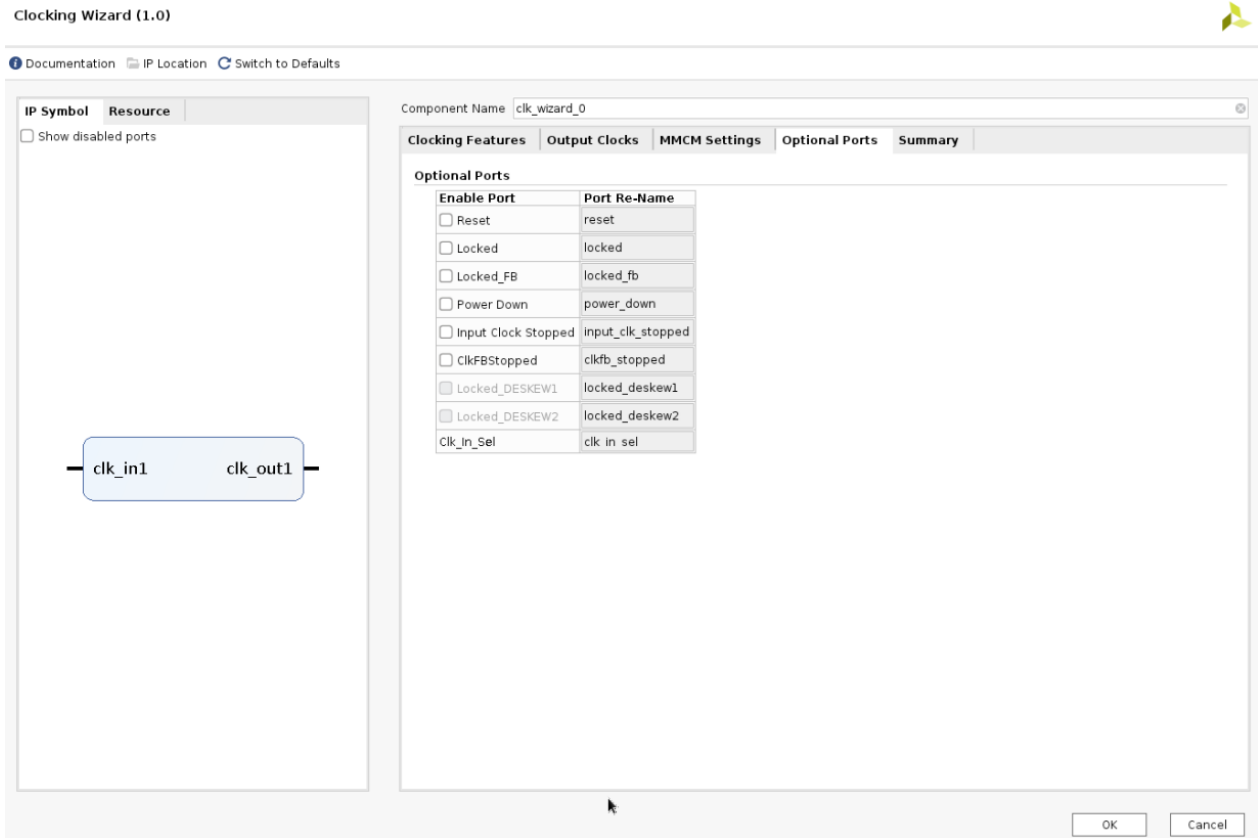
Figure 16: Primitive Override



Optional Ports

The first summary page, as shown in the following figures, displays summary information about the input and output clocks. This information is also provided as comments in the generated source code and in the provided XDC.

Figure 17: Optional Ports



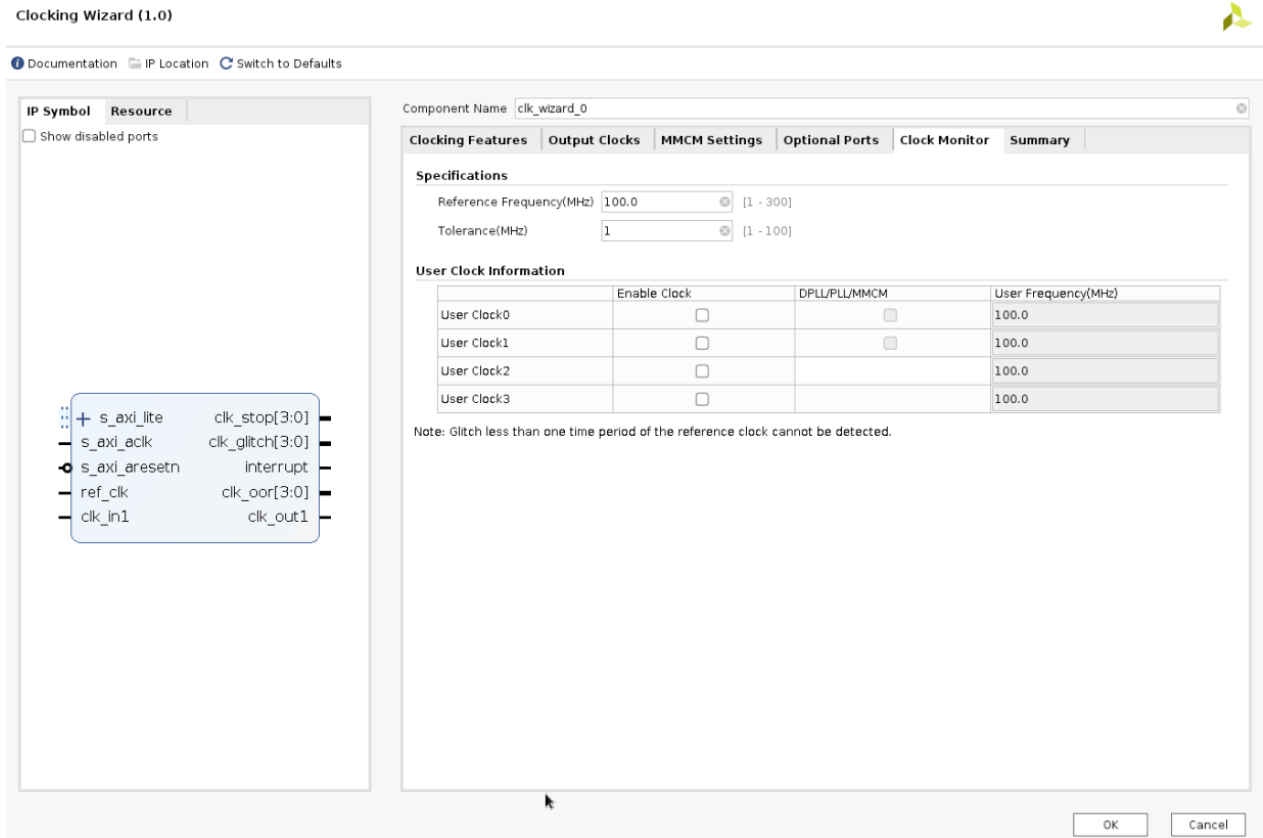
Selecting Optional Ports

All other optional ports that are not handled by selection of specific clocking features are listed under Optional Inputs/Outputs. Click to select the ports that you wish to make visible; inputs that are unused are tied off appropriately, and outputs that are unused are labeled as such in the provided source code.

Clock Monitor

The Clock Monitor feature allows you to monitor the clocks in a system; typically, the inputs to the MMCM/PLL/DPLL. It detects changes in the frequency of the clock, glitches in the clock, or a clock stop. It also gives you the option to specify the tolerance required. For example, if you want to see an error only if the frequency is 1 MHz higher than requested, a tolerance of 1 MHz must be specified as shown in the following figure.

Figure 18: Clock Monitor



- **Clock Stop:** The Clock Stop goes High when the clock is flat-lined.
- **Clock Glitch:** The Clock Monitor can detect a glitch in the user clock. The minimum glitch it can detect in the user clock is one clock period of the reference clock.

Note: The Clock Glitch condition might overlap with the Clock Overrun.
- **Clock Out of Range:** The Clock Monitor detects if the user clock frequency exceeds or goes below the required frequency.
- **Reference Clock Frequency:** The reference clock frequency determines the frequency of the clock to be monitored.
- **Channel Clock Frequency:** You can choose the frequency of the clock to be monitored based on the value of reference clock.
- **Tolerance:** You can program the precision required to monitor the clock.
- **Enable_PLL/MMCM (0-1):** Enabling this option monitors the input clock to the MMCM/PLL/DPLL.

Note:

- If the Enable_PLL/MMCM options are enabled in the IDE, ensure that the primary/secondary clock frequency does not exceed 300 MHz.
- s_axi* ports will be available when clock monitor is selected, and interrupt status can be read through axi registers.
- Overrun and Underrun are termed as out-of-range errors.
- Only integer values of tolerance are accepted.
- Only clock frequencies (USER_CLK_FREQ_0/1/2/3) less than or equal to 300 MHz is accepted.
- **Clock Monitor Registers:** The Clock Monitor error status register, the Interrupt status register, and the Interrupt enable register have the following bit map:

Table 7: Clock Monitor Register Bit Map

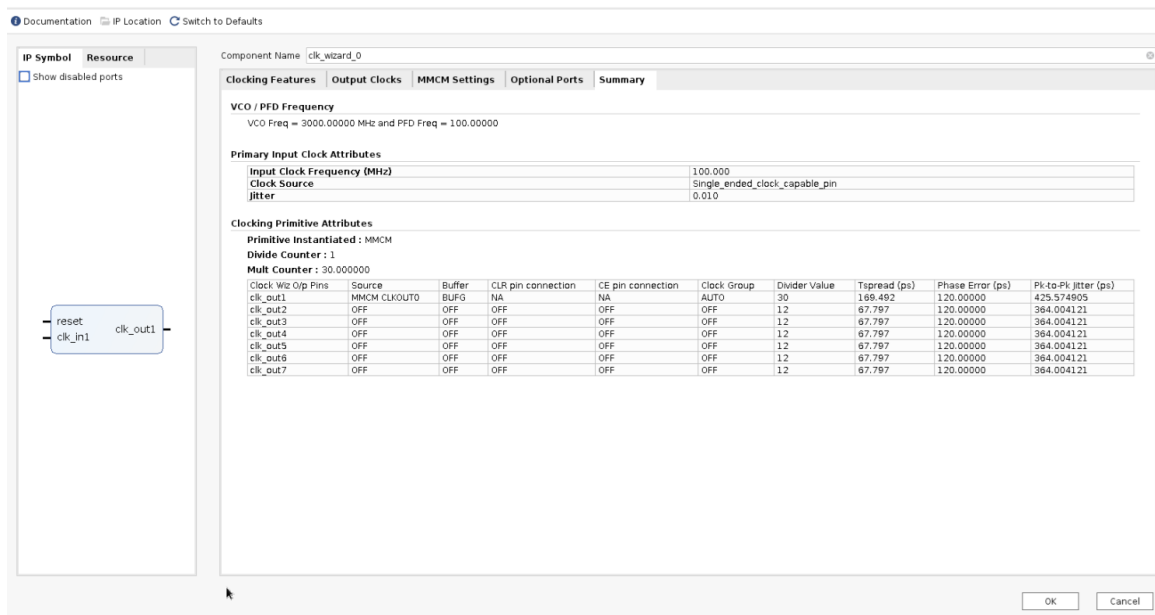
Bit Number	Description
0	User clock 0 frequency is greater than the specifications.
1	User clock 1 frequency is greater than the specifications.
2	User clock 2 frequency is greater than the specifications.
3	User clock 3 frequency is greater than the specifications.
4	User clock 0 frequency is lesser than the specifications.
5	User clock 1 frequency is lesser than the specifications.
6	User clock 2 frequency is lesser than the specifications.
7	User clock 3 frequency is lesser than the specifications.
8	Glitch occurred in user clock 0.
9	Glitch occurred in user clock 1.
10	Glitch occurred in user clock 2.
11	Glitch occurred in user clock 3.
12	Clock stop on user clock 0.
13	Clock stop on user clock 1.
14	Clock stop on user clock 2.
15	Clock stop on user clock 3.
16-31	Undefined

Summary

The summary page, shown in the following figure, contains general summary information. The Summary page gives you the information about the connections between the wizard output clocks and the primitive output clocks. The source column in the last table specifies the primitive output pin. It states whether the clock is generated directly from the primitive or derived using MBUFGCE. The Buffer column in the last table specifies the buffer that drives the output clock. IP will infer the buffer optimally in case of Auto buffer selected by user. The Clock Group in the last table specifies the clock group selected for that output clock. The CLR Pin connection in the last table will give CLR Pin information for each clock. This information is significant when

BUFGCE_DIV or MBUFGCE are selected explicitly and also when Auto Buffer ("Buffer" or "Buffer with CE") is inferred from MBUFGCE. The CE pin connection in the table will give the information of CE pin for each clock. This information is significant when BUFGCE or BUFGCE_DIV or MBUFGCE or "Buffer with CE" is selected. Divider value, Tspread, Phase Error and Pk-to-Pk Jitter columns in the table will give the information on the divider value, Tspread, Phase Error and Pk-to-Pk Jitter for each clock out respectively. Tspread is the actual spread, as calculated in [Spread Spectrum](#).

Figure 19: Summary Screen



Resource Estimate Summary

A resource estimate is provided based on the chosen clocking features.

Input Clocking Summary

Information entered on the first page of the GUI is shown for the input clocks.

Output Clocking Summary

Derived timing information for the output clocks is shown. If the chosen primitive has an oscillator, the VCO frequency is provided as reference. If you have a secondary input clock enabled, you can choose which clock is used to calculate the derived values.

Port Names

The Clocking Wizard allows you to name the ports according to their use. If you want to name the HDL port for primary clock input, simply type in the port name in the adjacent text box. The text boxes contain the default names. In the case of primary clock input, the default name is CLK_IN1.



IMPORTANT! Be careful when changing the port names as it could result in syntax errors if the port name entered is any reserved word of VHDL or Verilog or if that signal is already declared in the module.

Deskew Feature Support for Versal

The Clocking Wizard provides a drop-down menu for each clock-out for you to select DESKEW_PD1 or DESKEW_PD2 in PI Control column. Based on your choice, the CLKOUTx_PHASE_CTRL is set accordingly to 2'b01 (for network 1/PD1) or 2'b11 (for network 2/PD2) or 2'b10 for Fine_PS or 2'b00 for None. The following table describes primitive and number of deskew network details:

Table 8: Primitive Vs. Deskew Network

Primitive	Number of DESKEW Networks	Dyn_PS Option
MMCM	2	DESKEW_PD1 and DESKEW_PD2
XPLL (PLL)	2	DESKEW_PD1 and DESKEW_PD2
DPLL	1	DESKEW_PD1
AUTO**	2	DESKEW_PD1 and DESKEW_PD2

Notes:

1. AUTO primitive can be MMCM/DPLL/MBUFGCE depending on the selected PI Control option.

• Deskew Connection implementations and Prerequisites:

- In any primitive, `clk_in_deskew` and `clkfb_deskew` must have the same frequency
- `clkfb_deskew` connection is in the feedback path from one of the clock out (which is selected as deskew)
- The buffer in the feedback path can be either BUFG, BUFGCE, BUFGCE_DIV or MBUFGCE buffer
- In the case of **No_Buffer** selected for the clock out that wants to be connected to `clkfb_deskew`, then the `clkfb_deskew` port gets exposed and the connection to `clkfb_deskew` must be provided by you with any buffer based on the requirement
- In DPLL, `clk_in` and `clk_in_deskew` should always be tied
- In MMCM or XPLL, `clk_in_deskew` connection must be made as MUX between `clk_in` signal [It can be `clk_in1` or `clk_in2` from DESKEWx_IN option] and any of the clock output (in which case any value from `clk_out1` to `clk_out7` can be chosen for DESKEWx_IN) and external clock (`clk_ext` to be chosen as DESKEWx_IN).

- Whenever DESKEW_x_IN is chosen as `clk_ext` which means `clk_in1_deskew/clk_in2_deskew` will be exposed as port and the connection will be made to `clk_in_deskew` port of primitive. User must ensure the frequency of the `clk_in_deskew` to be same as `clkfb_deskew` even in the case of external clock selection.

The Clocking Wizard core provides an interface to select programmable delay for deskew network 1 or 2. Based on your choice, DESKEW_DLY_EN_x/DESKEW_DLY_PATH_x, DESKEW_DELAY_x and DESKEW_x_IN/DESKEW_x_FB attributes are set accordingly. You can program the delay in number of taps in the range provided by the wizard. DESKEW1_IN, DESKEW1_FB, DESKEW2_IN and DESKEW2_FB values are used to connect `clk_in1_deskew`, `clkfb1_deskew`, `clk_in2_deskew` and `clkfb2_deskew` respectively. The range of DESKEW_x_IN is from 0 to 9 (`clk_in1`, `clk_out` <1-7>, `clk_ext`), which means, if you want to connect `clk_inx_deskew` to primary input clock, you need to select it as 0 (`clk_in1`) and if it is secondary input clock, you need to select it as 9 (`clk_in2`), and you need to select as 8 (`clk_ext`) if you want to connect `clk_inx_deskew` with external/outside `clk_in_deskew` and if it is `clk_out` <1-7> then values <1-7> can be selected. The range of DESKEW_x_FB is from 1 to 7 which means, if `clkfbx_deskew` needs to be connected from any of the clock out you need to provide respective value. If it is `clk_out` <1-7> then values <1-7> has to be provided. There are few limitations set by the wizard to configure deskew networks on different clock outputs. The following are some examples:

- Consider selecting three clock outputs such as `clk_out1` = 100 MHz, `clk_out2` = 200 MHz, and `clk_out3` = 250 MHz. If you select **PI Control** for `clk_out1` to be DESKEW_PD1, then there is no option of DESKEW_PD1 for `clk_out3` as the frequency of `clk_out3` is not integer multiple/divider of `clk_out1`. Whereas both the options of DESKEW_PD1 and DESKEW_PD2 can be chosen for `clk_out2` as the frequency of `clk_out2` is integer multiple/divider of `clk_out1`.
- Similarly, if you select **PI Control** for `clk_out2` to be DESKEW_PD2, then neither DESKEW_PD1 nor DESKEW_PD2 option is provided for `clk_out3` as the frequency of `clk_out3` is different from `clk_out2` and `clk_out1` and also the frequency is neither integer multiple/divider of `clk_out1` nor that if `clk_out2`.
- DESKEW_PD2 is provided as an option for `clk_out3` if DESKEW_PD2 is not selected for `clk_out2`.

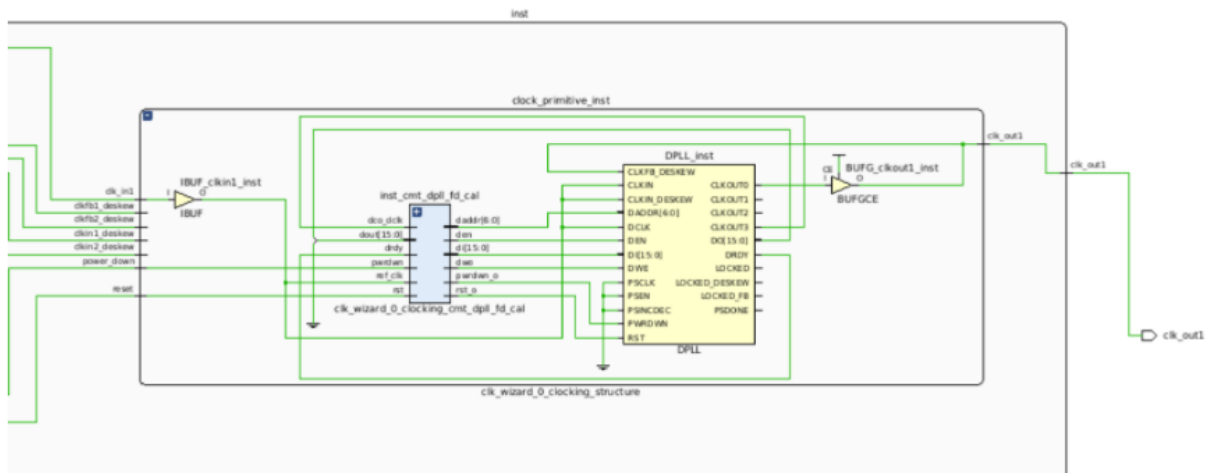
Note: For more information on Deskew, see Chapter-4 in *Versal ACAP Clocking Resources Architecture Manual* ([AM003](#)).

Digital Deskew for DPLL

The clocking wizard provides a drop-down menu for each `clk_out` starting from `clk_out` 1 to 3 for you to select DESKEW_PD1 from the PI control column. When you select this option, 2'b01 is set to CLKOUT_x_PHASE_CTRL parameter.

When you select DPLL deskew feature, `reset` and `power_down` ports are exposed automatically and these signals must pass through a calibrated soft logic present inside the IP. Whenever the soft logic is enabled, `PERF_MODE` of the primitive is set to `FULL` by the IP. Then the outputs of the calibrated logic are connected to the DPLL primitive. Clock input connection to the DPLL must be a `ref_clk` connection on the calibrated soft logic. This feature/calibration logic implementation helps you to get the proper clock output without any glitches. Following is the schematic with digital deskew in DPLL enabled:

Figure 20: Digital Deskew Enabled Schematic



Dynamic Reconfiguration through AXI4-Lite

The Clocking Wizard core provides an AXI4-Lite interface for the dynamic reconfiguration of the clocking primitive MMCM/XPLL/DPLL. This interface is enabled when Dynamic Reconfiguration is enabled. Mixed language RTL is delivered by the core when AXI4-Lite interface is used.

 **IMPORTANT!** This feature is not supported when Spread Spectrum is enabled and also when Digital Deskew for DPLL is enabled.

Register Space provides details of the signals of AXI4-Lite and the tables in the [Register Space](#) section provide details of the clock configuration registers.

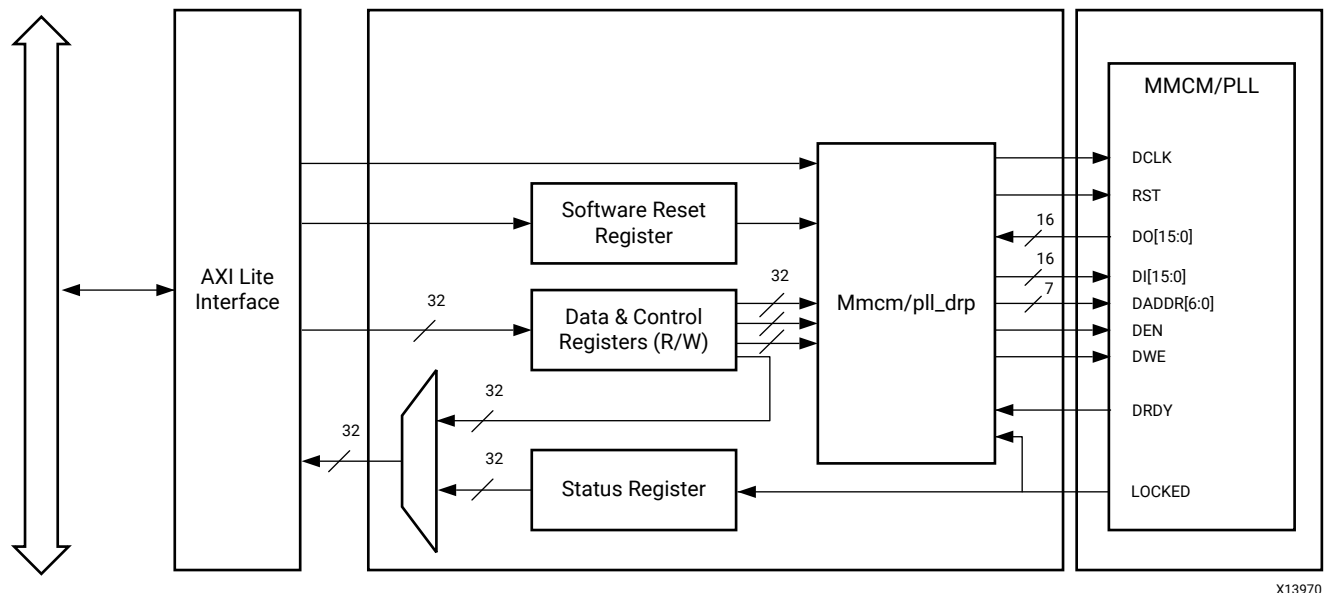
The Clocking Wizard core uses a configuration state machine listed in and extends from two fixed state configuration to program any valid range of multiply, divide, phase, and duty cycle. In this state machine, State 1 corresponds to default state configured through Clocking Wizard interface. State 2 corresponds to user-configuration loaded into the clock configuration register detailed in the tables in [Register Space](#). State 2 values are also initialized with the State 1 values so that a valid configuration is stored by default. All the dynamic reconfiguration registers are to be updated whenever you want to reprogram the clock.

Perform the following steps for dynamic reconfiguration:

1. Generate the Clocking Wizard IP enabling dynamic reconfiguration.
2. Open another Clocking Wizard with the same input clock and the features as intended.
3. Now, change the output clock features in the output clock tab of the Vivado® IDE as required for dynamic reconfiguration.
4. Generate the IP.
5. A file with name `<component_name>_drp_address_map` gets generated in the IP sources area. This file has the address and the data which needs to be written into AXI interface for reconfiguring.
6. After writing all the registers in the file, now write address `0x014` with `0x03` as mentioned in the register map to initiate the reconfiguration.

An example of this feature has been described in the following section.

Figure 21: Dynamic Reconfiguration using AXI4-Lite Interface



X13970

Example for Dynamic Reconfiguration through AXI4-Lite

The input and output clock frequencies are 100 MHz in the Clocking Wizard by default.

1. The output clock frequency needs to be reconfigured to 50 MHz with a phase shift of 90 degrees.
2. The file gets generated in the sources area as shown below.

```
00000330 1600
00000334 0f0f
00000338 5a07
0000033c 0f0f
00000340 0a00
00000344 0303
00000348 0a00
0000034c 0303
00000350 0a00
00000354 0303
00000358 002f
0000037c 0000
00000380 0000
00000384 0000
00000390 0000
00000394 0000
00000398 0e80
0000039c 41db
000003a0 43e9
000003a8 0002
000003c0 0001
000003cc 0a00
000003d0 0303
000003d4 0a00
000003d8 0303
000003dc 0a00
000003e0 0303
000003e4 0000
000003e8 0000
000003f0 0000
000003f8 1380
000003fc 0001
```

3. The file specifies all the AXI Registers with the address and the data which needs to be written into it. Configure all the AXI Registers with respect to the table.
4. Configure the address: C_BASEADDR + 0x014 with 0x00000003 to set the LOAD and SEN bits.
5. Wait for the locked signal. The new frequency can be checked at `clkout1` output port.

Note:

1. DRP interface is not supported for Versal® devices, APB3 interface is used internally. For details on DRP/APB3 ports, see *Versal ACAP Clocking Resources Architecture Manual* ([AM003](#)).
2. While reconfiguring, even reconfiguring for one clock output, all other active clock outputs will be reset.

Auto Buffer Selection and Clock Grouping

- In auto mode (the clock out drives set to "Buffer"), all those clock outs must be driven through one of the buffers (the buffer can be BUFG or MBUFGCE).
- In auto mode (the clock out drives set to "Buffer with CE"), all the clock outs are driven through one of the buffers with CE input (BUFGCE or MBUFGCE).
- IP consider all the clock outs where drives selected as "Buffer" or "Buffer with CE" into auto buffer criteria and which buffer driven through is chosen optimally based on other settings provided by you.
- MBUFGCE will be inferred if the clock outs are having frequencies as /1, /2, /4 and /8 in addition to phase set to 0 degree and duty cycle set to 50% and all those clock out to be under same clock group.
- Each clock out are provided with five clock group options to choose from
 - Auto- Default option
 - CLOCK_A
 - CLOCK_B
 - CLOCK_C

- None
- Auto clock grouping means you do not want to provide any clock grouping explicitly, but IP will make the clocks in to separate groups optimally depending on the MBUFGCE inference
- At max three MBUFGCE be inferred as auto buffer (as at max seven clocks are present, each MBUFGCE inference to have at least two clocks)
- For all the clocks selected with explicit clock grouping irrespective of auto buffer selection will be grouped together and the constraints gets generated (CLOCK_DELAY_GROUP constraints will be generated in <ip>.xdc file)

Following are the example combinations with auto buffer (drives selected as "Buffer") and auto clock grouping:

1. When all the frequencies are independent to each other (no integer division between any of them) then all clock out are inferred with separate BUFG.
2. When all the frequencies are integer divides of clk_out1 as in following table

Table 9: Frequencies with Integer Divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	3
Clk_out4	4
Clk_out5	5
Clk_out6	6
Clk_out7	8

- Two MBUFGCEs (MBUFGCE1: clkout1,2,4,7 and MBUFGCE2: clkout3,6) and remaining clock (clkout6) with separate BUFG
 - clk1_clr_n is available as clk_out1 is the input to MBUFGCE1 and clk3_clr_n also available as clk_out3 is the input to MBUFGCE2
3. When some of the frequencies are same as clk_out1 and some are integer divides of clk_out1

Table 10: Frequencies with same as clk_out1 and Integer Divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	1
Clk_out3	1
Clk_out4	1
Clk_out5	2
Clk_out6	3

Table 10: Frequencies with same as clk_out1 and Integer Divides of clk_out1 (cont'd)

Clk_out	DIV
Clk_out7	4

- Three clocks (clkout1,5,7) can be derived from one MBUFGCE and remaining clocks (clkout2,3,4,6) are derived from separate BUFG.
 - clk1_clr_n is available as clk_out1 is the input to MBUFGCE.
4. When only four clock outs are present and all are integer divides of clk_out1

Table 11: Four Clocks with /1, /2, /4 and /8 of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	4
Clk_out4	8

- MMCM primitive should not be inferred in this case and IP infer MBUFGCE
 - clk1_clr_n is available as clk_out1 is the input to MBUFGCE
5. When the frequencies are divided by 1 or 2

Table 12: Frequencies with /1 and /2 of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	1
Clk_out4	2
Clk_out5	1
Clk_out6	2
Clk_out7	1

- Three MBUFGCEs (MBUFGCE1: clkout1,2; MBUFGCE2: clkout3,4 and MBUFGCE3: clkout5,6) and remaining clock (clkout7) is inferred through BUFG
 - clk1_clr_n is available as clk_out1 is the input to MBUFGCE1, clk3_clr_n is available as clk_out3 is the input to MBUFGCE2 and clk5_clr_n is available as clk_out5 is the input to MBUFGCE3
6. When all the frequencies are even integer divides of clk_out1

Table 13: Frequencies with even Integer Divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	2
Clk_out4	4
Clk_out5	4
Clk_out6	6
Clk_out7	8

- Two MBUFGCE (MBUFGCE1: clkout1,2,4,7 and MBUFGCE2: clkout3,5) should be used to infer 6 clock outs and remaining clock (clkout6) should be inferred from BUFG
- clk1_clr_n is available as clk_out1 is the input to MBUFGCE1 and clk3_clr_n also available as clk_out3 is the input to MBUFGCE2
- Sample test case with clock grouping is selected:** When all the frequencies are integer divides of clk_out1

Table 14: Frequencies of Integer Divides of clk_out1 and with Multiple Clock Grouping

Clk_out	DIV	CLK GROUP
Clk_out1	1	CLOCK A
Clk_out2	2	CLOCK B
Clk_out3	3	Auto
Clk_out4	4	CLOCK A
Clk_out5	5	Auto
Clk_out6	6	Auto
Clk_out7	8	CLOCK B

- Three MBUFGCE (MBUFGCE1: clkout1, 4: MBUFGCE2: clkout2, 7 and MBUFGCE3: clkout3, 6) and remaining clock (clkout5) is inferred through BUFG.
- clk1_clr_n is available as clk_out1 is the input to MBUFGCE1, clk2_clr_n is available as clk_out2 is the input to MBUFGCE2 and clk3_clr_n is available as clk_out3 is the input to MBUFGCE3.

Following are the example combinations with Auto Buffer (Drives selected as “Buffer with CE”) and Auto Clock Grouping

- When all the frequencies are independent of each other (no integer division between any of them), all clock outs are inferred with separate BUFGCE
- When all the frequencies of integer divides of clk_out1 as in below table

Table 15: Frequencies of integer divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	3
Clk_out4	4
Clk_out5	5
Clk_out6	6
Clk_out7	8

- Two MBUFGCEs (MBUFGCE1: clkout1,2,4,7 and MBUFGCE2: clkout3,6) and the remaining clock (clkout6) with separate BUFGCE
 - clk1_clr_n will be available as clk_out1 is the input to MBUFGCE1, and clk3_clr_n also available as clk_out3 is the input to MBUFGCE2
 - clk1_ce will be available as clk_out1 is the input to MBUFGCE1, clk3_ce also available as clk_out3 is the input to MBUFGCE2, and clk_out5_ce is also available as BUFGCE for clk_out5
3. When some of the frequencies are same as clk_out1, and some are integer divides of clk_out1

Table 16: Frequencies are same as clk_out1, and some of integer divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	1
Clk_out3	1
Clk_out4	1
Clk_out5	2
Clk_out6	3
Clk_out7	4

- Three clocks (clkout1,5,7) can be derived from one MBUFGCE, and the remaining clocks (clkout2,3,4,6) will be derived from separate BUFGCE.
 - clk1_clr_n and clk1_ce will be available as clk_out1 is the input to MBUFGCE
 - clk_out2_ce, clk_out3_ce, clk_out4_ce, and clk_out6_ce are also available as 4 separate BUFGCE inferred.
4. When only four clock outs are used and all are integer divides of clk_out1

Table 17: Four clock out are used and all are integer divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	4
Clk_out4	8

- All four clocks can get from one MBUFGCE
 - clk1_clr_n and clk1_ce will be available as clk_out1 is the input to MBUFGCE
5. When the frequencies are divided by 1 or 2

Table 18: Frequencies are divided by 1 or 2

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	1
Clk_out4	2
Clk_out5	1
Clk_out6	2
Clk_out7	1

- Three MBUFGCEs (MBUFGCE1: clkout1,2; MBUFGCE2: clkout3,4 and MBUFGCE3: clkout5,6) and the remaining clock (clkout7) is inferred through BUFGCE
 - clk1_clr_n, clk1_ce will be available as clk_out1 is the input to MBUFGCE1, clk3_clr_n, clk3_ce will be available as clk_out3 is the input to MBUFGCE2 and clk5_clr_n, clk5_ce will be available as clk_out5 is the input to MBUFGCE3
 - clk_out7_ce is available as BUFGCE inferred for clk_out7
6. When all the frequencies are even integer divides of clk_out1

Table 19: Frequencies are even integer divides of clk_out1

Clk_out	DIV
Clk_out1	1
Clk_out2	2
Clk_out3	2
Clk_out4	4
Clk_out5	4
Clk_out6	6
Clk_out7	8

- Two MBUFGCE (MBUFGCE1: clkout1,2,4,7 and MBUFGCE2: clkout3,5) should be used to infer 6 clock outs and the remaining clock (clkout6) should be inferred from BUFGCE
- clk1_clr_n, clk1_ce will be available as clk_out1 is the input to MBUFGCE1 and clk3_clr_n, clk3_ce also available as clk_out3 is the input to MBUFGCE2
- clk_out6_ce is available as BUFGCE will be inferred for clk_out6

Sample test case with clock grouping is selected:

When all the frequencies are integer divides of clk_out1

Table 20: Frequencies of integer divides of clk_out1

Clk_out	DIV	CLK GROUP
Clk_out1	1	CLOCK A
Clk_out2	2	CLOCK B
Clk_out3	3	Auto
Clk_out4	4	CLOCK A
Clk_out5	5	Auto
Clk_out6	6	Auto
Clk_out7	8	CLOCK B

Three MBUFGCEs (MBUFGCE1: clkout1,4; MBUFGCE2: clkout2,7 and MBUFGCE3: clkout3,6) and the remaining clock (clkout5) is inferred through BUFGCE

- Three MBUFGCE (MBUFGCE1: clkout1, 4; MBUFGCE2: clkout2, 7 and MBUFGCE3: clkout3, 6) and the remaining clock (clkout5) is inferred through BUFGCE.
- clk1_clr_n, clk1_ce will be available as clk_out1 is the input to MBUFGCE1, clk2_clr_n, clk2_ce will be available as clk_out2 is the input to MBUFGCE2 and clk3_clr_n, clk3_ce will be available as clk_out3 is the input to MBUFGCE3.
- clk_out5_ce is available as BUFGCE inferred for clk_out5

Note: In all the above cases, the *_clr_n pin connected to MBUFGCE is active-Low signal and you should tie to 1 for MBUFGCE to be active.

User Parameters

The following table shows the relationship between the fields in the Vivado® IDE and the user parameters (which can be viewed in the tool command language (Tcl) console).

Table 21: User Parameter Description

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Component Name	COMPONENT_NAME	Clk_wizard_0

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value		User Parameter/Value	Default Value
Primitive Range: MMCM, PLL, DPPLL, Auto, and None		PRIMITIVE_TYPE	MMCM
Phase Alignment Range: TRUE, FALSE		USE_PHASE_ALIGNMENT	FALSE
Minimize Power Range: TRUE, FALSE		USE_MIN_POWER	FALSE
Jitter_Optimization Range:		JITTER_SEL	Balanced
Display Name	Actual Value		
Balanced	No_Jitter		
Minimize Output Jitter	Min_O_Jitter		
Maximize Input Jitter filtering	Max_I_Jitter		
Secondary Range: TRUE, FALSE		USE_INCLK_SWITCHOVER	FALSE
Port Name		PRIMARY_PORT, SECONDARY_PORT	PRIMARY_PORT Clk_in1 SECONDARY_PO RT Clk_in2
Primary Input Clock (MHz) Range (Default): 10-1230 (range vary depending on the part/board type selected)		PRIM_IN_FREQ	100.000
Secondary Input Clock (MHz) Range (Default): 72-144		SECONDARY_IN_FREQ	100.000
Jitter Options Range: UI, PS		JITTER_OPTIONS	UI
Primary Input Jitter Range: PS:[10-999]UI:[0.001-0.10]		CLKIN1_JITTER	0.010
Secondary Input Jitter Range: UI:[0.001-0.10] PS:[10-999]		CLKIN2_JITTER	0.010
Source Range: 1-Single ended Clock Capable pin 2-Differential ended Clock capable pin 3-Global Buffer 4-No Buffer		PRIM_SOURCE	Single-ended clock capable pin

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value		User Parameter/Value	Default Value
Source Range: 1-Single ended Clock Capable pin 2-Differential ended Clock capable pin 3-Global Buffer 4-No Buffer		SECONDARY_SOURCE	Single-ended clock capable pin
Clk_out<1-7> Range: True, False		CLKOUT_USED	TRUE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE
Port Name Range: True, False		CLKOUT_PORT	clk_out1, clkout2, clk_out3, clk_out4, clk_out5, clk_out6, clk_out7
Requested Output clock frequency		CLKOUT_REQUESTED_OUT_FREQ	100.000, 100.000, 100.000, 100.000, 100.000, 100.000, 100.000
Requested Phase Range: -360:360		CLKOUT_REQUESTED_PHASE	0.000, 0.000,0.000, 0.000, 0.000, 0.000, 0.000
Requested Duty Cycle Range: 0.001:99.99		CLKOUT_REQUESTED_DUTY_CYCLE	Default Value: 50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 50.0
Drives Range: BUFG, BUFGCE, BUFGCE_DIV, MBUFGCE, Buffer, Buffer with CE, and No_buffer		CLKOUT_DRIVES	BUFG, BUFG,BUFG, BUFG, BUFG, BUFG, BUFG_GT
PI Control Range: DESKEW_PD1, DESKEW_PD2, Fine_PS, and None		CLKOUT_DYN_PS	None, None, None, None, None, None, None
Clock Grouping Range: Auto, CLOCK_A, CLOCK_B, CLOCK_C and None		CLKOUT_GROUPING	Auto, Auto, Auto, Auto, Auto, Auto, Auto
Source Range:		FEEDBACK_SOURCE	Automatic Control On-Chip
Display Name	Actual Parameter		
Automatic Control On-Chip	FDBK_AUTO		
Automatic Control Off-Chip	FDBK_AUTO_OFFCHIP		
User-Controlled On-Chip	FDBK_ONCHIP		
User-Controlled Off-Chip	FDBK_OFFCHIP		

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Signaling Range: - Single-ended: SINGLE - Differential: DIFF	CLKFB_IN_SIGNALING	Single-ended
Reset Range: True, False	USE_RESET	False
Powerdown Range: True, False	USE_POWER_DOWN	False
Input clk stopped Range: True, False	USE_INCLK_STOPPED	False
Locked Range: True, False	USE_LOCKED	False
Locked_FB Range: True, False	USE_LOCKED_FB	False
Locked_DESKEW1 Range: True, False	USE_LOCKED_DESKEW1	False
Locked_DESKEW2 Range: True, False	USE_LOCKED_DESKEW2	False
deskew1_delay_en Range: True, False	DESKEW1_DELAY_EN	False
deskew2_delay_en Range: True, False	DESKEW2_DELAY_EN	False
deskew1_delay_path Range: ClkIn Path, ClkFb Path	DESKEW1_DELAY_PATH	ClkFb Path
deskew1_delay Range: 0-63	DESKEW1_DELAY	0
deskew2_delay_path Range: ClkIn Path, ClkFb Path	DESKEW2_DELAY_PATH	ClkFb Path
deskew2_delay Range: 0-63	DESKEW2_DELAY	0
Clkfbstopped Range: True, False	USE_CLKFB_STOPPED	False
Reset Type Range: active-High, active-Low	RESET_TYPE	active-High
Phase shift Mode Range: WAVEFORM, LATENCY	PHASESHIFT_MODE	LATENCY
Allow Override Mode Range: True, False	OVERRIDE_PRIMITIVE	False
Bandwidth Range: MMCM: LOW, HIGH, OPTIMIZED PLL: LOW, HIGH, OPTIMIZED DPLL: OPTIMIZED	BANDWIDTH	OPTIMIZED
CLKFBOUT_MULT Range: Depends on the primitive selected	CLKFBOUT_MULT	30

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
CLKFBOUT_PHASE Range: (-360.000:360.00)	CLKFBOUT_PHASE	0.000
COMPENSATION Range: DPLL: INTERNAL AUTO, BUF_IN, EXTERNAL PLL: INTERNAL MMCM: AUTO, INTERNAL, BUF_IN, EXTERNAL	COMPENSATION	AUTO
DIVCLK_DIVIDE Range: Depends on the primitive selected	DIVCLK_DIVIDE	1
CLKFBOUT_PHASE_CTRL Range: None, Fine_PS, DESKEW_PD1, DESKEW_PD2	CLKOUTFB_PHASE_CTRL	None
Divide	CLKOUT<1-7>_DIVIDE	30
Duty Cycle Range: 0-100	CLKOUT<1-7>_DUTY_CYCLE	0.500
Phase Range: -360:360	CLKOUT<1-7>_PHASE	0.000
Locked	LOCKED_PORT	Locked
Clk_in_sel	CLK_IN_SEL_PORT	Clk_in_sel
Reset	RESET_PORT	reset
Input_clk_stopped	INPUT_CLK_STOPPED_PORT	Input_clk_stopped
Clkfb_stopped	CLKFB_STOPPED_PORT	Clkfb_stopped
Locked_FB	LOCKED_FB_PORT	locked_fb
Locked_DESKEW1	LOCKED_DESKEW1_PORT	locked_deskew1
Locked_DESKEW2	LOCKED_DESKEW2_PORT	locked_deskew2
Power_Down	POWER_DOWN_PORT	power_down
Dynamic Reconfig Range: TRUE, FALSE	USE_DYN_RECONFIG	FALSE
Spread Spectrum Range: TRUE, FALSE	USE_SPREAD_SPECTRUM	FALSE
Enable Clock Monitoring Range: TRUE, FALSE	ENABLE_CLOCK_MONITOR	FALSE
ZHOLD Range: TRUE, FALSE	ZHOLD	FALSE
Ss Mode Range: - CENTER HIGH - CENTER LOW - DOWN HIGH - DOWN LOW	SS_MODE	CENTER HIGH
Modulation Freq Range: 25-250	SS_MOD_FREQ	250

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Dynamic Reconfig Interface Options Range: Enable_AXI, Enable_APB3	INTERFACE_SELECTION	Enable_AXI
Enable User Clock0 Range: TRUE, FALSE	ENABLE_SUER_CLOCK0	FALSE
Enable User Clock1 Range: TRUE, FALSE	ENABLE_SUER_CLOCK1	FALSE
Enable User Clock2 Range: TRUE, FALSE	ENABLE_SUER_CLOCK2	FALSE
Enable User Clock3 Range: TRUE, FALSE	ENABLE_SUER_CLOCK3	FALSE
User clk0 Freq Range: 1-300	USER_CLK_FREQ0	100
User clk1 Freq Range: 1-300	USER_CLK_FREQ1	100
User clk2 Freq Range: 1-300	USER_CLK_FREQ2	100
User clk3 Freq Range: 1-300	USER_CLK_FREQ3	100
Enable PLL0 Range: TRUE, FALSE	Enable_PLL0	FALSE
Enable PLL1 Range: TRUE, FALSE	Enable_PLL1	FALSE
Ref clk Freq Range: 1-300	REF_CLK_FREQ	100
Precision Range: 1-100	PRECISION	1
Relative InclK Range: REL_PRIMARY, REL_SECONDARY	RELATIVE_INCLK	REL_PRIMARY
Deskew1 IN Range: 0-9	DESKEW1_IN	0
Deskew1 FB Range: 1-7	DESKEW1_FB	1
Deskew2 IN Range: 0-9	DESKEW2_IN	0
Deskew2 FB Range: 1-7	DESKEW2_FB	1
Safe Clock Startup Range: TRUE, FALSE	USE_SAFE_CLOCK_STARTUP	FALSE
Safe Clock Startup Mode Range: DESKEW_MODE, BUFGCE_MODE	SAFECLOCK_STARTUP_MODE	DESKEW_MODE
CE TYPE Range: SYNC, ASYNC, HARDSYNC	CE_TYPE	SYNC
BUFGCE DIV CE TYPE Range: SYNC, HARDSYNC	BUFGCE_DIV_CE_TYPE	SYNC

Table 21: User Parameter Description (cont'd)

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
CE and CLR SYNC Circuit External to Core Range: FALSE, TRUE	CE_SYNC_EXT	FALSE
Use Clock Sequencing Range: FALSE, TRUE	USE_CLOCK_SEQUENCING	FALSE
CLKOUT<1-7>_SEQUENCE_NUMBER Range: 1-7	CLKOUT<1-7>_SEQUENCE_NUMBER	1

Notes:

- Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Constraining the Core

Required Constraints

At least one clock constraint is required for period and jitter. The following command signifies the setting of +/- 100 ps (= 0.1 ns) peak to peak jitter on the primary clock port propagating through input port `clk_in1`:

```
create_clock -period 10.0 [get_ports clk_in1]
set_input_jitter [get_clocks -of-objects [get_ports clk_in1]] 0.1
```

The core level XDC has early processing order so core level XDC constraints are applied first and then are overridden by the user-provided constraints.

Device, Package, and Speed Grade Selections

Supports all packages, speed grades and devices.

Clock Frequencies

Clock Management

The core can generate a maximum of seven output clocks with different frequencies.

Clock Placement

No clock placement constraint is provided.

Banking

Bank selection is not provided in xdc file.

I/O Standard and Placement

No I/O or placement constraints are provided.

Simulation

For comprehensive information about Vivado® simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

You can simulate the example design using the `open_example_project` flow in Vivado® design tools.

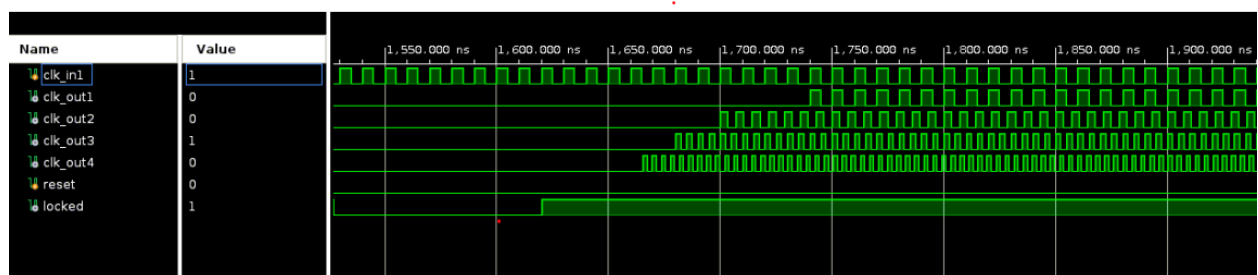
If you open an example project, then the simulation scripts are generated in the working directory in:

```
example_project/<component_name>-example/<component_name>-example.sim/sim_1/
```

Simulation Waveforms for the Safe Clock Startup Feature without sequencing

Simulation, when Safe Clock Startup is true, is illustrated in the below snapshot.

Figure 22: Simulation, when Safe Clock Startup is true



The sequencing is disabled in the above configuration.

After LOCKED is high, clk_out1 will enable after 8 cycles to first clock out the frequency of primitive. Similarly, after LOCKED is high, clk_out2 will enable after 8 cycles to second clock out the frequency of primitive.

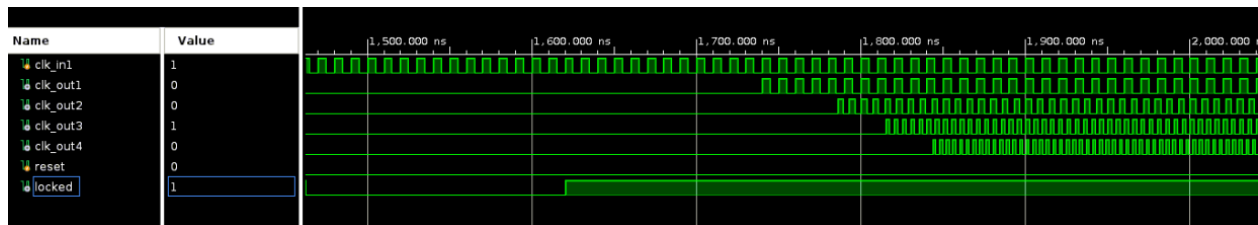
Similarly, after LOCKED is high, clk_out3 will enable after 8 cycles to third clock out frequency of primitive.

Similarly, after LOCKED is high, clk_out4 will enable after 8 cycles to fourth clock out frequency of primitive.

Simulation Waveforms for the Safe Clock Startup Feature with sequencing

The below figure illustrates simulation when Safe Clock Startup is true and Use Clock Sequencing is true, with the required sequence number in the table (Consider sequence is 1,2,3,4).

Figure 23: Simulation, when Safe Clock Startup is true and Use Clock Sequencing



After LOCKED is high, clk_out1 will enable after 8 cycles to the first clock out the frequency of primitive. And after CLKOUT1 is high, clk_out2 will enable after 8 cycles to the second clock out the frequency of primitive.

After CLKOUT2 is high, clk_out3 will enable after 8 cycles to the third clock out frequency of primitive.

After CLKOUT3 is high, clk_out4 will enable after 8 cycles to the fourth clock out frequency of primitive.

This will be useful if the user uses a case where the clocks need to come in sequence.

Detailed Example Design

In Vivado® design tools, the `open_example_project [get_ips <component_name>]` parameter in tcl console invokes a separate example design project where it creates `<component_name>_exdes` as top module for synthesis and `<component_name>_tb` as top module for simulation. You can run implementation or simulation of the example design from example project.

Unsupported Features in Example Design

Example design is not supported for:

- DPLL primitive when Deskew is used, and "LOCKED_FB" is disabled. Enable "LOCKED_FB" for generating example design.
- DPLL primitive when both Deskew and Dynamic reconfiguration is enabled. Disable either of them to generate an example design.
- An example design is not supported when Clock Monitor is enabled and Primitive as None.

Directory and File Contents

The `open_example_project [get_ips <component_name>]` parameter creates the `example_project` directory in the working area.

Example design contains the counters on all the output clocks and MSBs of these counters are used as output to observe on LEDs on board.

The following file describes the example design for the Clocking Wizard core.

Verilog

```
<project_name>/<project_name>.srcs/sources_1/ip/<component_name>/
example_design/<component_name>_exdes.v
```

The top-level example designs adds clock buffers where appropriate to all of the input and output clocks. All generated clocks drive counters, and the high bits of each of the counters are routed to a pin. This allows the entire design to be synthesized and implemented in a target device to provide post place-and-route gate-level simulation.

Note: Example design files are delivered only in Verilog.

Test Bench

This chapter contains information about the provided test bench in the Vivado® Design Suite environment.

The following file describes the demonstration test bench.

Verilog

```
<project_name>/<project_name>.srcs/sources_1/ip/<component_name>/simulation/  
<component_name>_tb.v
```

The demonstration test bench is a simple Verilog program to exercise the example design and the core. It does frequency calculation and check of all the output clocks. It reports all the output clock frequency and if any of the output clocks is not generating the required frequency, it reports an error.

Note: Test bench files are delivered only in Verilog.

Migrating

This information is provided to assist those designers who are experienced with the DCM and PLL Architecture Wizards. It highlights the differences between the old and new cores.

Migrating to the Vivado Design Suite

For information about migrating to the Vivado® Design Suite, see *ISE to Vivado Design Suite Migration Guide* ([UG911](#)).

Note: You cannot migrate to a Versal® Clocking Wizard from a Non-Versal® Clocking Wizard as they both are two separate IPs. A new IP must be built from scratch with similar properties. .

For example, a user has a requirement of two output clocks with `clk_out1` as 120 MHz and `clk_out2` as 150MHz and a separate phase/duty cycle requirement for two clocks. In Non-Versal, the user will pass a value for each clock attribute, whereas, in Versal, the parameters must be provided as a string for all the output clocks when using batch mode. For example, if you want to feed the requested clock frequencies for the clocks, the parameter `CLKOUT_REQUESTED_OUT_FREQ` must be assigned a value of 120, 150, 100, 100, 100, 100, and 100. (100 is the default value) It is the combination of all output frequencies from 1 to 7. If the string does not contain 7 values, it fills the initial frequencies, and for the rest, it takes the default value. Similarly, for all other parameters of `CLKOUT_DRIVES`, `CLKOUT_DYN_PS`, `CLKOUT_MATCHED_ROUTING`, `CLKOUT_REQUESTED_DUTY_CYCLE`, `CLKOUT_REQUESTED_PHASE`, `CLKOUT_USED`, and `CLKOUT_PORT`, you must pass the respective values as a list of values in batch mode instead of a single value for each clock.

`PRIMITIVE`, which can be “MMCM”, `PLL`, `Auto`” in Non-Versal, can be mapped to the `PRIMITIVE_TYPE` parameter.

`OVERRIDE_MMCM`, which can set to “True/False” should map to the `OVERRIDE_PRIMITIVE` parameter.

Differences between the Clocking Wizard and the Legacy DCM and PLL Wizards

There are several changes to the GUI and the wizard use model as described in the following subsections.

Primitive Selection

The old wizards required you to choose the correct GUI (DCM or PLL) before configuring the desired primitive.

The new Clocking Wizard automatically selects the appropriate primitive and configures it based on desired parameters. You can choose to override this choice in the event that multiple primitives are available, as is the case for the Spartan®-6 device family.

Symbol Pin Activation

The old wizards had a symbol with clickable pins to enable a port.

For the new Clocking Wizard, the symbol shows the ports that are currently active. To enable a port, enable the appropriate feature in the GUI. For example, enabling the secondary input clock enables the `CLK_IN2` and `CLK_IN_SEL` ports and activates those ports in the symbol.

Parameter Override

The new Clocking Wizard allows you to override any calculated parameter within the wizard by switching to override mode.

Port Display Conventions

The new Clocking Wizard displays the superset of ports covering all device families. Ports that are not available for the selected target device are dimmed out. Information on the legal ports for a specific primitive can be found in the device family-specific FPGA or clocking resources on the [Xilinx Support web page](#).

Visibility of Clock Ports

The new Clocking Wizard provides a clocking network that matches the your requirements rather than making clock ports visible. As a result, your clock names will not match the exact names for the primitive. For example, while the “first” clock available for the MMCM is `CLKOUT0`, the highest priority clock available to you is actually named `CLK_OUT1`.



IMPORTANT! *This change in numbering is especially important to consider if parameter overriding is desired.*

GUI Information Gathering Order

Some of the information-gathering ordering has changed. For the new Clocking Wizard, the general flow is:

1. Select the clocking features.
2. Configure the input clock parameters.

3. Configure the output clock parameters.
4. Choose feedback and optional ports.
5. View (and optionally override) calculated parameters.
6. Final summary pages.

For cascading clocking components, non-buffered input and output clocks are available for easy connection.

Debugging

This appendix includes details about resources available on the Xilinx Support website and debugging tools.

Finding Help on Xilinx.com

To help in the design and debug process when using the core, the [Xilinx Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Xilinx Community Forums](#) are also available where members can learn, participate, share, and ask questions about Xilinx solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [Xilinx Support web page](#) or by using the Xilinx® Documentation Navigator. Download the Xilinx Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Solution Centers

See the [Known Issues](#) for support on known issues.

See the [Xilinx Solution Centers](#) for support on devices, software tools, and intellectual property at all stages of the design cycle. Topics include design assistance, advisories, and troubleshooting tips.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with a Xilinx product. Answer Records are created and maintained daily ensuring that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [Xilinx support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

For clock uncertainty issue due to memory resource activity, see AR [76369](#).

Technical Support

Xilinx provides technical support on the [Xilinx Community Forums](#) for this LogiCORE™ IP product when used as described in the product documentation. Xilinx cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Xilinx Community Forums](#).

Debug Tools

There are many tools available to address Clocking Wizard design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The Vivado® Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in Xilinx® devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)

- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The Vivado® debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If your outputs go to 0, check your licensing.

Known Issues

- Run connection automation in IPI is still under development.
- Phase parameter does not propagate to the Port when OVERRIDE mode is set in IPI.
- When jitter value is switched between PS and UI, the resulted values are sometimes incorrect.

Debug for Dynamic Reconfiguration

If the configuration of your Clocking Wizard is not as per the required dynamic clock, ensure to verify the following:

- The VCO frequency falls into the valid range for the given multiply and the divide values.
- The fractional enable bit must be set only for the non-integer values.

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Xilinx Support](#).

References

These documents provide supplemental material useful with this guide:

1. *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))
 2. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
 3. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
 4. *Versal ACAP Clocking Resources Architecture Manual* ([AM003](#))
 5. *Versal ACAP Hardware, IP, and Platform Development Methodology Guide* ([UG1387](#))
 6. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
 7. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
 8. *Vivado Design Suite User Guide: Design Analysis and Closure Techniques* ([UG906](#))
 9. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
-

Training Resources

1. [Vivado Design Suite Hands-on Introductory Workshop](#)
2. [Vivado Design Suite Tool Flow](#)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
10/26/2022 Version 1.0	
Output Clock Ports	Added additional buffer BUFG_GT.
04/20/2022 Version 1.0	
Digital Deskew for DPLL	Added new section
Status and Control Ports	Updated Clocking Wizard Status and Control Ports table note
Clock Monitor	Updated Clock Stop section
06/30/2021 Version 1.0	
Applications	Updated with additional information
Clock Monitor	Updated with additional information
Summary	Updated with additional information
Figure 19: Summary Screen	Updated image
Deskew Feature Support for Versal	Added new feature
Auto Buffer Selection and Clock Grouping	Updated the content
Input Clock Ports	Added new ports to the list
Master Answer Record for the Core	Updated new Answer Record
02/02/2021 Version 1.0	
Feature Summary	Added Safe Clock Startup.
Figure 2	Updated figure.
Table 2	Updated table.
Clocking Features	Added new features.
Clock Monitor	Added new features.
Table 6	Updated table.
Figure 5	Updated figure.
Figure 6	Updated figure.
Summary	Added new features.
Auto Buffer Selection and Clock Grouping	Added information about "Buffer with CE."
Safe Clock Startup	Added new section.
User Parameters	Added new features.
Simulation	Added new features.
Unsupported Features in Example Design	Added new unsupported features.
07/14/2020 Version 1.0	
Initial release.	N/A

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2020-2022 Advanced Micro Devices, Inc. Xilinx, the Xilinx logo, Alveo, Artix, Kintex, Kria, Spartan, Versal, Vitis, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. All other trademarks are the property of their respective owners.